UNIVERSITE DU BURUNDI (UB)

Institut de Statistique Appliquée

Master 1, en Informatique décisionnelle

Cours de Logiciels de calcul formel

Pr. NIYONSABA Thérence

A/A 2024-2025

0. Introduction

Data science(science des données):rencontre big data+analyse de ces données=> En tirer conclusion.

Actuellement, nous observons une augmentation des données et l apparition des données non structurées.

Les bases de données relationnelles utilisent une structure de données organisée en tables, avec des relations définies entre les tables. Le langage SQL est utilisée pour formuler les requêtes d interrogation a la base des données relationnelles.

Les bases de données non relationnelles, également appelées bases de données NoSQL, ont une structure de données plus flexible que les bases de données relationnelles.

Elle peut stocker sous différents formats des données (clés-valeurs, documents, graphes, etc.).

Les requêtes NoSQL sont utilisées pour interroger les bases des données non relationnelles.

Les bases de données non relationnelles sont adaptées pour les applications qui nécessitent une grande scalabilité et une forte performance pour gérer des volumes élevés de données en temps réel.

0.1 Rappel sur la commande SELECT

- SELECT colonne FROM table WHERE conditions
- SELECT DISTINCT
- SELECT …. FROM…. WHERE condition1 AND condition1
- SELECT …. FROM…. WHERE (condition1 AND condition1) OR (condition1 AND condition1)
- SELECT …. FROM…. WHERE nom colonne BETWEEN « valeur1 » AND « valeur 2 ». On peut utiliser NOT BETWEEN(si on veut avoir des valeurs qui ne sont pas situes entre cette intervalle)
- SELECT …. FROM…. WHERE nom colonne BETWEEN « valeur1 » AND « valeur 2 ». On peut utiliser NOT BETWEEN(si on veut avoir des valeurs qui ne sont pas situes entre cette intervalle)
- Operateur LIKE:

Exemple:

SELECT …. FROM…. WHERE colonne LIKE informatique

SELECT …. FROM…. WHERE colonne LIKE ‘%e’

SELECT …. FROM…. WHERE colonne LIKE ‘e%’

SELECT …. FROM…. WHERE colonne LIKE ‘%e%’

- La commande GROUP BY, Il sert a grouper plusieurs résultats et utiliser une fonction de totaux sur un groupe de résultat.

Exemple:

SELECT Client, SUM(Prix) FROM Achat GROUP BY client

0.2 LES JOINTURES

La jointure permet de lier deux ou plusieurs tables a conditions qu'il existe au moins un lien entre ces tables.

Un lien est presenté par une clef primaire dans une table et une clef etrangère dans l'autre table.

Syntaxe generale des jointures

Les noms des differentes relations sont separés par des virgules.

SELECT table1.attribut1,table2.attribut2,etc.

FROM table1 **INNER JOIN** table2 **ON** table1.attribut.CP

= table2.attributCE;

WHERE conditions

- la commande **INNER JOIN**, aussi appelée EQUIJOIN, est un type de jointures très communes pour lier plusieurs tables entre-elles.Syntaxe générale(diapo précédant)
- La commande **CROSS JOIN** est un type de jointure sur 2 tables SQL qui permet de retourner le produit cartésien
- la commande **LEFT JOIN** (aussi appelée LEFT OUTER JOIN) est un type de jointure entre 2 tables. Cette commande nous permet de lister tous les résultats de la table de gauche (left = gauche) même s'il n'y a pas de correspondance dans la deuxième tables.
- la commande **RIGHT JOIN** (ou RIGHT OUTER JOIN) est un type de jointure entre 2 tables qui permet de retourner tous les enregistrements de la table de droite (right = droite) même s'il n'y a pas de correspondance avec la table de gauche. Il y a la commande **FULL JOIN**

La clause ORDER BY

La clause ORDER BY permet de trier et ordonner le resultat. La syntaxe est la suivante :

ORDER BY expression [ASC | DESC] ;

ASC spécifie l'ordre ascendant et DESC l'ordre descendant du tri. En l'absence de précision ASC ou DESC, c'est l'ordre ascendant qui est utilise par défaut.

Exemple: liste des clients (№, nom, prenom, date de creation) par ancienneté:

```
SELECT №,nom,prenom,date_creation
FROM Client
ORDER BY date_creation ASC;
```

0.3 Les transactions

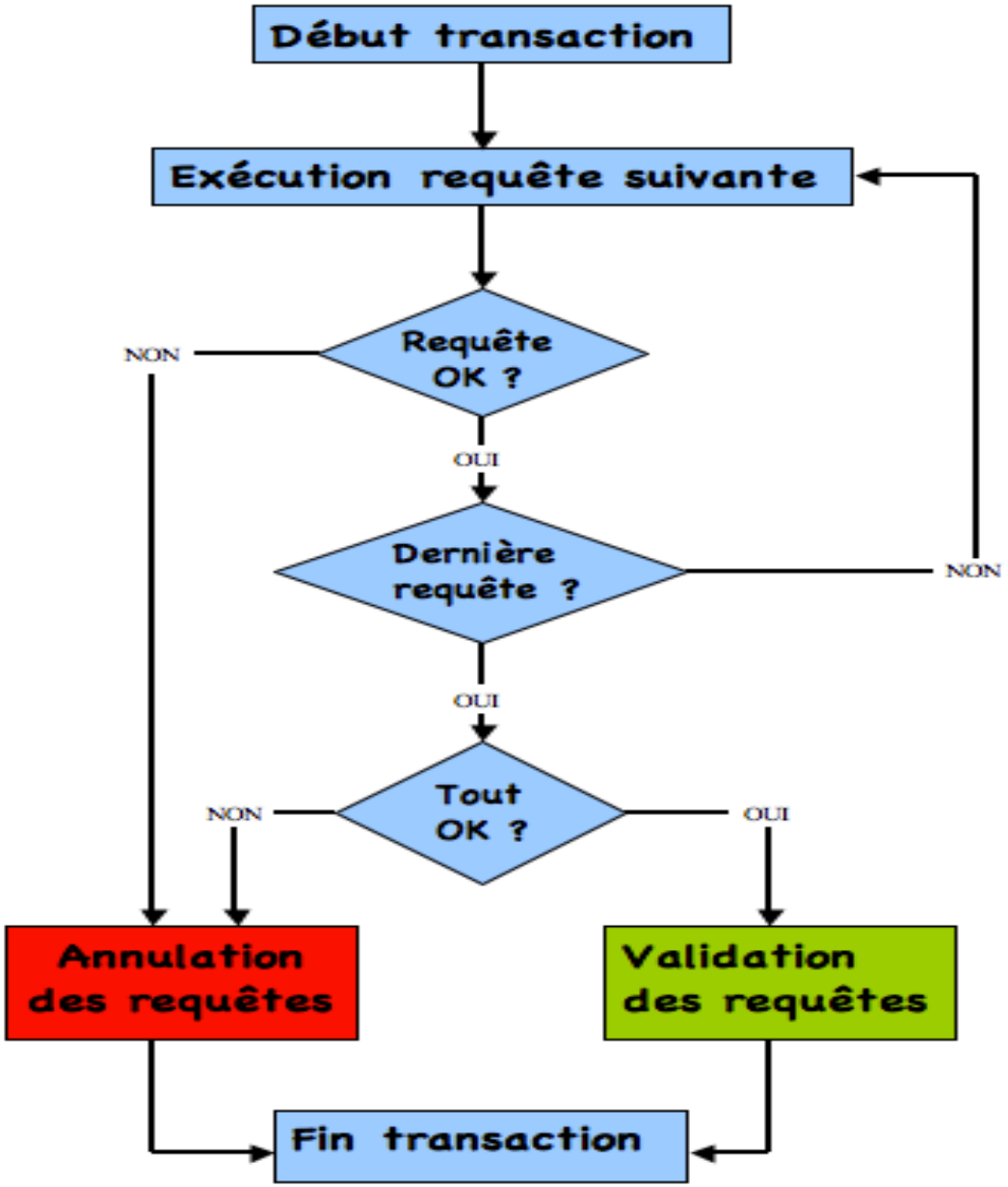
les transactions

Une transaction, c'est un ensemble de requêtes qui sont exécutées en un seul bloc. Ainsi, si une des requêtes du bloc échoue, le bloc de requêtes échoue.

Exemple de débit et crédit des comptes des clients de la banque.

Sans transactions, certaines opérations risqueraient d'être à moitié réalisées, et la moindre erreur, la moindre interruption pourrait avoir des conséquences énormes. En effet, les transactions permettent de regrouper des requêtes dans des blocs, et de faire en sorte que tout le bloc soit exécuté en une seule fois, cela afin de préserver l'intégrité des données de la base.

Déroulement d'une transaction



Syntaxe et utilisation

Quand on valide les requêtes d'une transaction, on dit aussi que l'on **commite** les changements. À l'inverse(l'annulation des requêtes) s'appelle un **rollback**.

Chaque requête effectuée dans MySQL est directement commitée (validée). Elle est automatiquement commitée. C'est donc en mode "autocommit".

Pour quitter ce mode, il faut deactiver l'autocommit, selon la requête suivante : SET autocommit=0 ;

Exemple 1 : Validation des requêtes

```
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Baba', 5, '2012-02-13 15:45:00', 'F');
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Bibo', 5, '2012-02-13 15:48:00', 'M');
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Buba', 5, '2012-02-13 18:32:00', 'F');
-- Insertion de 3 rats bruns
UPDATE Espece
SET prix = 20
WHERE id = 5 ;
```

Les changements faits sont bien visibles. Les rats bruns valent maintenant 20 euros, et les trois nouvelles bestioles (Baba, Bibo et Buba) ont bien été insérées.

Exemple 2 : validation et annulation

Exécutez la séquence de requêtes suivante :

```
START TRANSACTION ;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Baba', 5, '2012-02-13 15:45:00', 'F');
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Bibo', 5, '2012-02-13 15:48:00', 'M');
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Buba', 5, '2012-02-13 18:32:00', 'F');
 -- Insertion de 3 rats bruns
COMMIT ;
UPDATE Espece
SET prix = 20
WHERE id = 5 ; -- Les rats valent 20 euros
ROLLBACK ;
```

Trois nouveaux rats bruns (les requêtes d’insertion ayant été validées), et ils ne valent toujours que l’ancien prix chacun.

Jalon de transaction

Lorsque l'on travaille dans une transaction, et que l'on constate que certaines requêtes posent problème, on n'a pas toujours envie de faire un rollback depuis le début de la transaction, annulant toutes les requêtes alors qu'une partie aurait pu être validée, on pose des jalons de transaction.

Il s'agit de points de repère, qui permettent d'annuler toutes les requêtes exécutées depuis ce jalon, et non toutes les requêtes de la transaction.

Syntaxe :

SAVEPOINT nom_jalon;

Exemple : .

```
START TRANSACTION ;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Popi', 5, '2007-03-11 12:45:00', 'M');
SAVEPOINT jalon1;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Momo', 5, '2007-03-12 05:23:00', 'M');
ROLLBACK TO SAVEPOINT jalon1;
INSERT INTO Animal (nom, espece_id, date_naissance, sexe)
VALUES ('Mimi', 5, '2007-03-12 22:03:00', 'F');
COMMIT ;
```

On n’utilise qu’une seule transaction, on valide à la fin. La seconde insertion n’a pas été faite au final, puisqu’elle a été annulée grâce au jalon. Seuls Popi et Mimi existent.

Pour supporter les transactions, une table doit être **transactionnelle**, ce qui, avec MySQL, est défini par le **moteur de stockage** utilisé pour la table.

Il existait différents moteurs de stockage possibles avec MySQL, dont les plus connus sont **MyISAM** et **InnoDB**. MyISAM ne supportant pas les contraintes de clés étrangères.

- les tables MyISAM sont non-transactionnelles, donc ne supportent pas les transactions ;
- les tables InnoDB sont transactionnelles, donc supportent les transactions.

ACID (**A**tomicité, **C**ohérence, **I**solation et **D**urabilité)

Ce sont les **critères** qu'un système utilisant les transactions doit respecter pour être **fiable.**

Atomicité: Une transaction doit être atomique, c'est-à-dire qu'elle doit former une entité complète et indivisible.

Chaque élément de la transaction, chaque requête effectuée, ne peut exister que dans la transaction.

Cohérence: Les données doivent rester cohérentes dans tous les cas, que la transaction se termine avec succès ou qu'elle soit interrompue. Tant que la totalité des étapes n'a pas été réalisée avec succès, les données restent dans leur état initial.

Isolation: Chaque transaction doit être isolée, donc ne pas interagir avec une autre transaction. Tous les transactions sont isoles pour éviter que l'execution de l'un n'interfère pas dans l'execution de l'autre. On isole donc les transactions grâce aux verrous.

Durabilité: Une fois la transaction terminée, les données résultant de cette transaction doivent être stockées de manière durable, et pouvoir être récupérées, en cas de crash du serveur par exemple.

0.4.Les contraintes des SGBD relationnels

La plupart des moteurs de SGBDs relationnels se base sur le concept de la transaction 4 qui est une unité d'action qui leur impose le respect des contraintes ACID acronyme de : **A**tomicité **C**ohérence ou consistance **I**solation **D**urabilité et qu'on nomme donc l'acidite de la transaction : ACID

0.5. Les Forces des SGBDs relationnels

Le succès des SGBDs relationnels est du essentiellement aux 12 règles de CODD et le modèle relationnel qui a permis de développer une vision de ce qu'est ou doit être une base de données.

Parmis ces nombreux avantages nous citons :

- **Cohérence transactionnelle forte** : garantissant la gestion de la concurrence, l'isolation entre les utilisateurs, la reprise sur panne.
- **Séparation logique et physique** : il y a une forte indépendance entre :
  - Modèle de données et structures de stockage
  - Requêtes déclaratives et exécution.

- **Structuration forte des données :** Simplicité de présentation, les données sont sauvegardées dans des tables.
- **Capacité a gérer des requêtes très complexes.**
- **Optimisation très fine des requêtes, index permettant un accès rapide aux données.**
- **Logiciels murs, stables, efficaces, riches en fonctionnalités et en interfaces.**
- **Contraintes d'integrite permettant d'assurer des invariants sur les données.**

0.6.Les limites des SGBDs relationnels

Bien que les SGBDs relationnels ont fait succès depuis plus de 20 ans. Néanmoins, un certain nombre de limitations importantes sont apparues. Les premiers acteurs qui ont rencontre ces limites furent les géants du web.

Le constat était simple : les SGBDs relationnels ne sont plus une solution pour un **environnement distribue requis par les volumes gigantesques de données,** ce n'est pas tant le langage SQL en lui même qui est inadapté mais les grands principes sur lesquels il a été construit (le modèle relationnel et transactionnel que nous avons vu tout en haut).

En effet, En Big Data, non seulement on doit gerer des quantités de données très importantes , mais aussi on veut un résultat très rapidement. Or Les SGBDs relationnels traditionnels atteignent ici leurs limites `a savoir:

Les structures de données : sont devenues de plus en plus complexes avec en contrepartie des moteurs de stockage évoluant peu. Le principal point faible des modèles relationnels est l'absence de gestion d'objets hétérogènes ainsi que le besoin de déclarer au préalable l'ensemble des champs représentant un objet.

Un système de jointure entre les tables : Concrètement un abus des jointures risque d'entrainer un traffic réseau conséquent accompagne d'un pic de consommation de la mémoire.

- **Contraintes d’integrite** : les SGBD relationnels sont limites `a des contrôles de cohérence simples. Il est donc difficile de modéliser des contraintes réelles correspondant aux données d’une entreprise.
- **Surcharge sémantique** : le modèle relationnel s’appuie sur un seul concept (la relation) pour modéliser `a la fois les entités et les associations (ou relations) entre ces entités. Il existe donc un décalage entre la réalité et sa représentation abstraite.
- **Les contraintes ACID en milieu distribue** : dans le cas des systèmes distribues, il est nécessaire de distribuer les traitements de données entre différents serveurs. Il devient alors difficile de maintenir les contraintes ACID `a l’échelle du système distribue entier tout en maintenant des performances correctes.

Chap.1 Problématique du Big Data

Contexte:

-Une grande partie de la population mondiale est connectée;-Internet des objets(smarts refrigellators, smarts watch,Smart TV…),

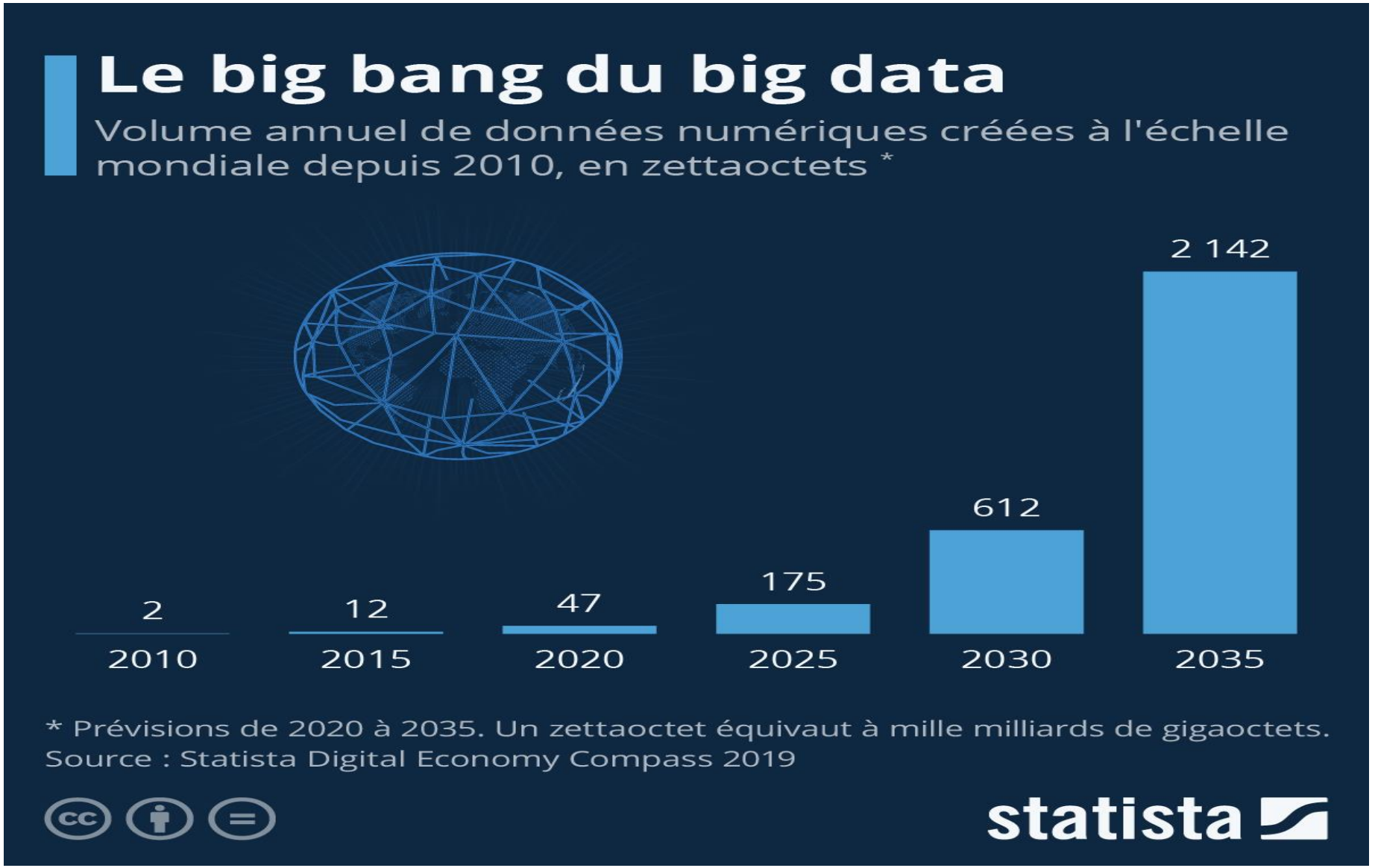
-Selon, https://fr.statista.com/ de 30 juillet 2020, Le volume annuel des données créées et échangées se multiplierons par trois ou quatre tous les cinq ans.

-Selon les estimation, 47 zettaoctets (mille milliard de Go) de données sont créées et échangées chaque année. Il faudrait se procurer 470 millions de disque durs (100 To), pour être en mesure de sauvegarder les 47 zettaoctets de données potentiellement créées chaque année.

Définition

Données massive(Megadonnées) représente les collections de données caractérisées par un volume, une vélocité et une variété si grands que leur transformation en valeur utilisable requiert l'utilisation de technologie et de méthodes analytiques spécifiques.

Volumes des données crées et échangées



De 3 V a 5V

La caractérisation de ces mégadonnées ou Big Data est généralement faite selon 3 « V », les V de Volume, de Variété et de Vélocité, auxquels s'ajoutent d'autres « V » complémentaires, comme ceux de Valeur et de Véracité/Validité.

Volume :volume massive

Variété: hétérogénéité, différentes ressources des données

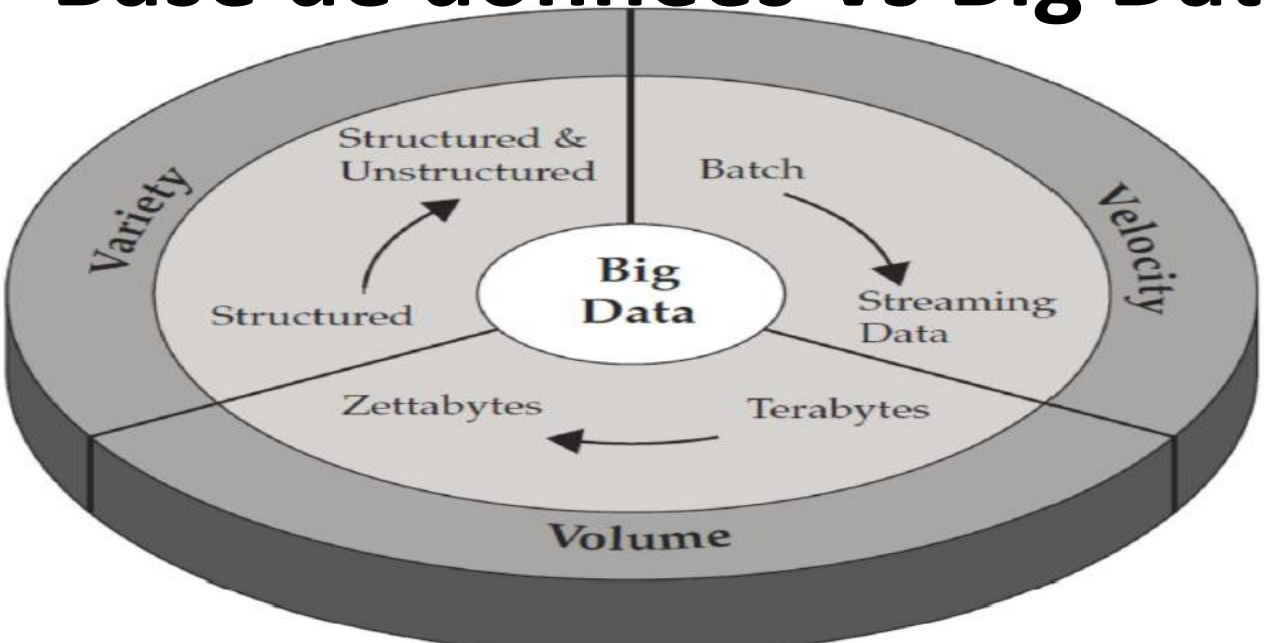
Vélocité : vélocité: aspect dynamique ou temporel des données et fait référence a leur délai d'actualisation et de traitement. Ce sont des données collectées en temps réel pour servir en temps réel aussi. Il faut les analyser en flux(streaming).

Valeur: On fait référence a la potentialité des données, en particulier en terme économique, l'analyse demande une certaine expertise statistique et science

Véracité ou Validité

Enfin, le caractère complémentaire « *véracité ou validité* » fait référence à la qualité des données et/ou aux problèmes éthiques liés à leur utilisation. Il comprend les problèmes de valeurs aberrantes ou manquantes (ces problèmes pouvant être résolus par le volume de données), mais aussi à la confiance que l'on peut avoir dans les données. S'il existe des critères permettant de qualifier la qualité des données, dans le cas de *big data*, cette vérification de la qualité est rendue difficile voire impossible du fait du volume, de la variété et de la vélocité spécifiques au Big Data.

Base de données Vs Big Data



Problématique de traitement et valorisation

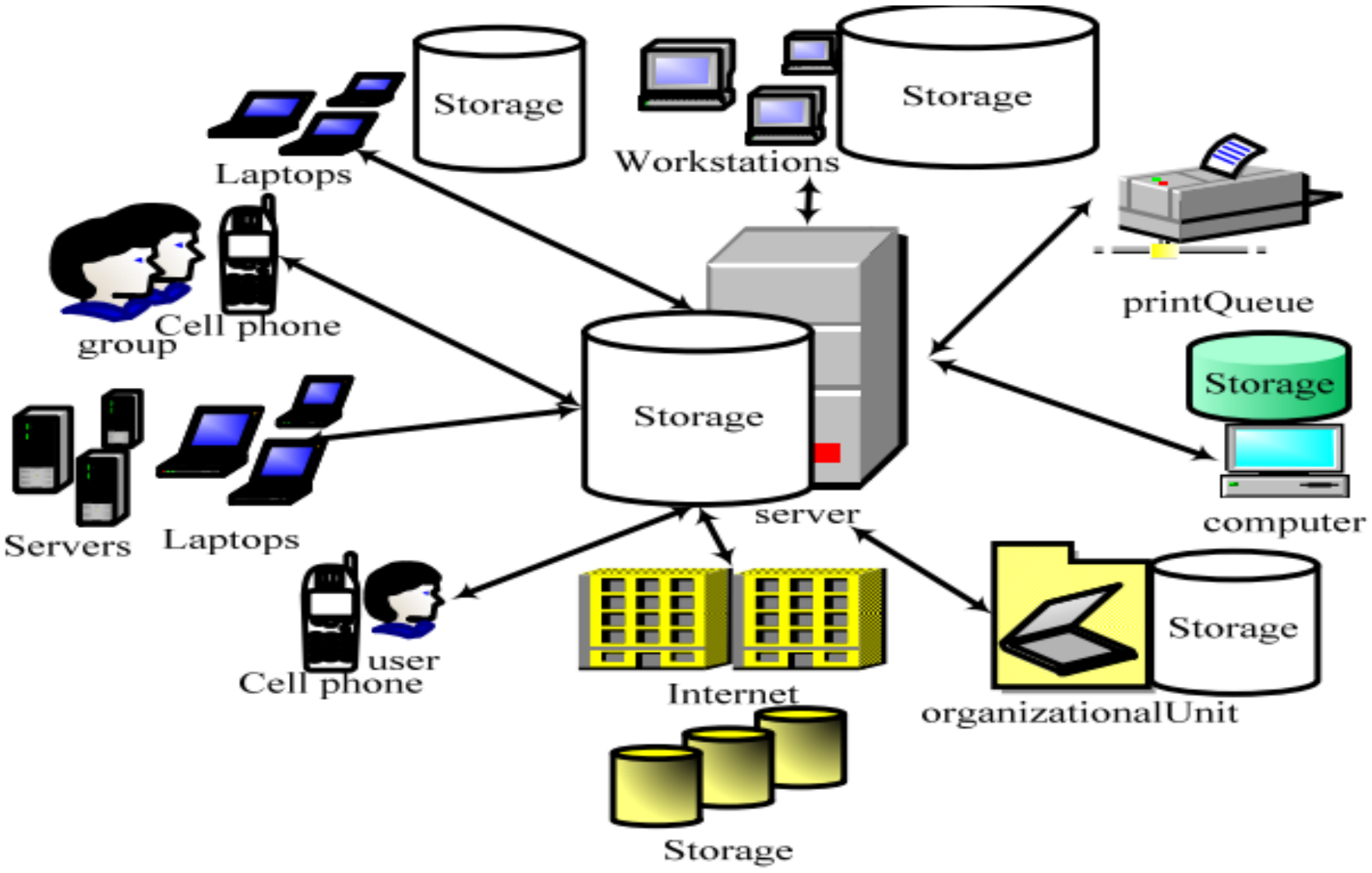
-Les données créées et échangées sont massives, données non relationnelles;

-Beaucoup de données sont dormantes, selon le journal, comarketing-news.fr/, 88% d'entre elles ne sont pas analysée et non exploitées;

-Trop de données, tue les données;

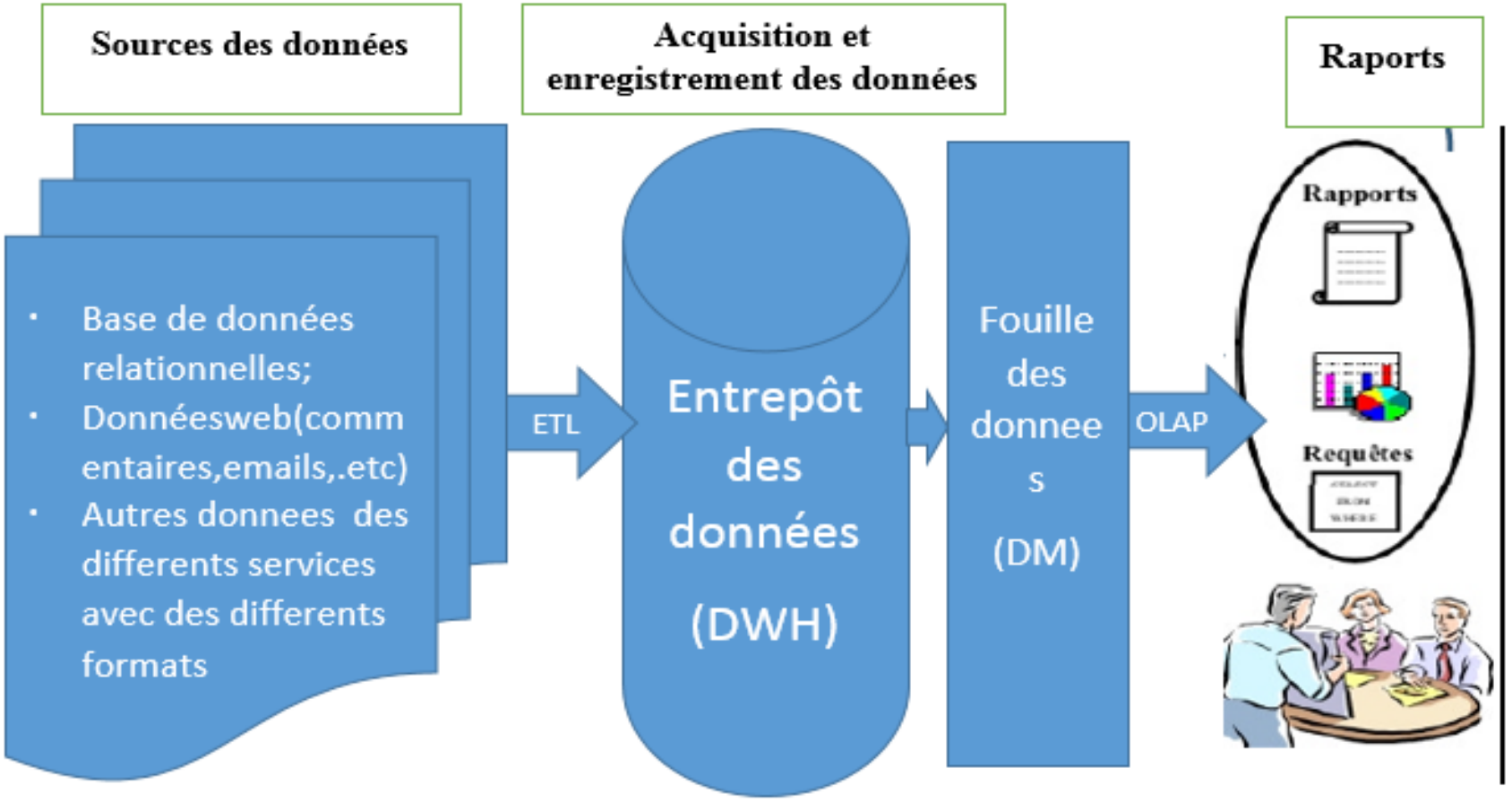
-Données de diffèrent format et volumineuses (exemple : Amazon, ebay, Lumitel, Econet Leo, banques…), les services des moteurs de recherche comme Google,…et autres;

Données hétérogènes issues de différentes sources



Sources hétérogènes des masses importantes des données

Principe de fonctionnement du système d extraction des connaissances a partir des données



E.T.L:Extract, Transform, Load
DWH: Data Ware House
DM: Datamining
OLAP: Online Analytical Processing

On rappelle que Data Mining est au cœur de KDD (Knowledge Data Discovery)

Solution proposée:

- Traitement parallèle(HALoop,BigSQL)
- Stockage parallèle(Multiserveurs:)

Leur traitement est distribue(sharding) sur différents serveur et utilise les algorithme de de Mapreduce pour le traitement distribue des megadonnees sur les différents serveurs.

Mapreduce est souvent intègre de le framework hadoop (Logiciel Opensource de la fondation Apache),dedies au stackage des donnees massives et ayant deux composant:

MapReduce est un environnement de programmation pour plateformes parallèles. Il suppose un système d'exploitation distribué gérant des données encodées sous forme de paires (c, v). Le premier élément de cette paire est appelé **clé**, et le second **valeur**.

1. Hadoop

Hadoop est un logiciel "open source" de la fondation Apache. C'est un environnement logiciel dédié au stockage et au traitement de larges volumes de données. Il est mis au point par Google, il permet de traiter de façon parallèle de gros volumes de données. Hadoop repose sur deux composantes essentielles :

- Un système de fichiers distribué (HDFS : Hadoop Distributed File System) : permet de manipuler les données comme si elles étaient dans un seul fichier.

=>Les données ne sont pas structurées et sont stockées sous forme de paires (clé,valeur) : un morceau de données correspond à une valeur et à chaque valeur est associé une clé.

- Une implémentation efficace de l'algorithme Mapreduce : permet de traiter les données en parallélisant les calculs.

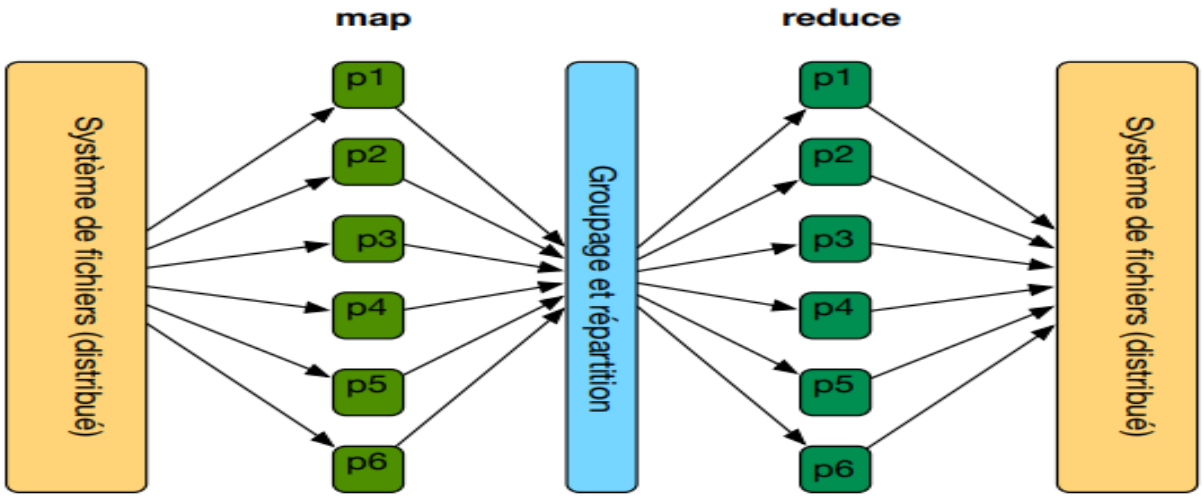
=> Le paradigme MapReduce est un modèle de programmation permettant de programmer simplement des calculs distribués dans une architecture Hadoop.

MapReduce propose essentiellement deux fonctions permettant à l'utilisateur de manipuler des paires clé-valeur : la fonction **map** et la fonction **reduce**.

La fonction map permet de créer un **ensemble de paires** clé-valeur à partir de toute paire clé-valeur. La fonction reduce permet de créer **une paire** clé-valeur à partir de toute paire clé-valeur dont la valeur regroupe l'ensemble des valeurs de **même clé** produite par map.

Fonction map : $(c, v) \rightarrow \{(c_1, v_1), \ldots, (c_k, v_k)\}$

Fonction reduce : $(c, \{v_1, \ldots, v_k\}) \rightarrow (c', v)$



L’utilisateur programme la **fonction map**, sous la contrainte de format imposée par **MapRedude**, c’est-à-dire la fonction map doit, pour chaque paire clé-valeur **(c, v),** créer un ensemble **{(c1, v1), . . . ,(ck, vk)}** de paires clé-valeur. A la suite de l’exécution de la fonction map sur chacun des processeurs, le système r**egroupe** toutes les valeurs produites par map partageant la même clé, créant ainsi un ensemble de paires **(c, {v1, . . . , vk}).** La fonction reduce, programmée par l’utilisateur, peut alors être appliquée sur les paires **(c, {v1, . . . , vk}),** afin de retourner des paires clé-valeurs.

Le facteur d’accélération et l’efficacité d’un programme MapReduce dépend beaucoup du nombre de clés que le système doit traiter lors de la phase de groupage et de répartition.

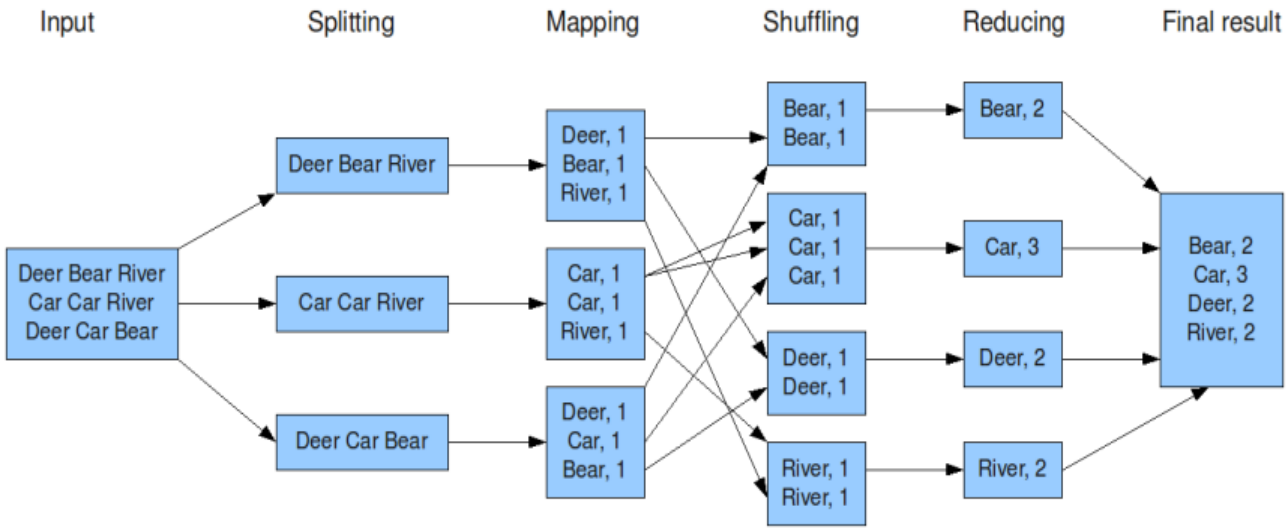
Une version (très simplifiée) d'un programme de type mapreduce est la suivante : mapreduce(input,map,reduce).

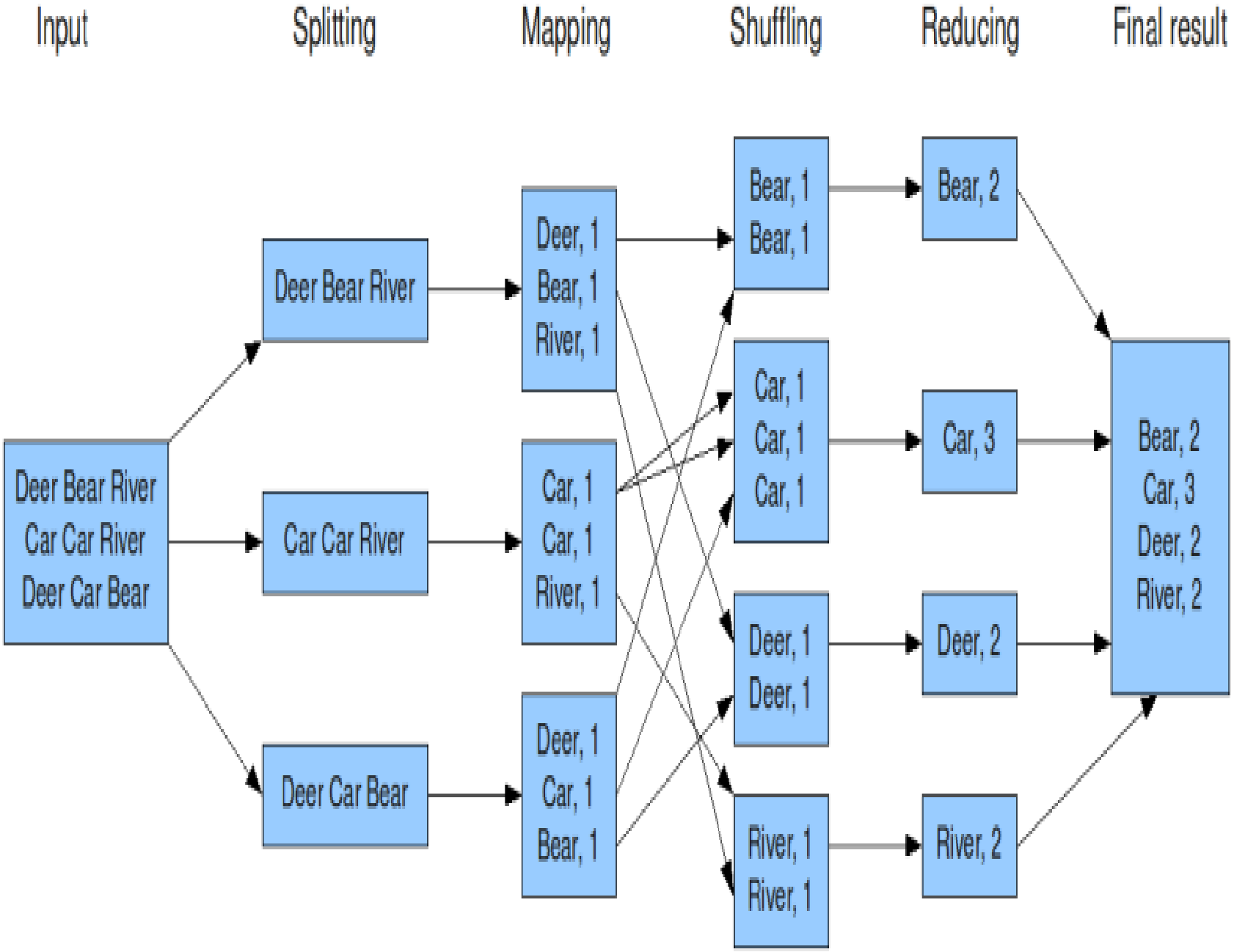
Les données (input) sont au format HDFS, elles sont donc découpées en blocs, dispatchées sur plusieurs machines.

• Sur chaque bloc est appliquée la fonction map qui prend en entrée des paires (clé,valeur) et renvoie des paires (clé,valeur).

• Les résultats sont remontés au niveau du noeud principal et regroupés par clés (shuffle).

• Pour chaque groupe de données possédant la même clé est appelé la fonction reduce() qui prend en entrée des paires (clé,valeur) et renvoie des paires (clé,valeur).




Input
Splitting
Mapping
Shuffling
Reducing
Final result
Deer Bear River
Car Car River
Deer Car Bear
Deer Bear River
Car Car River
Deer Car Bear
Deer, 1
Bear, 1
River, 1
Car, 1
Car, 1
River, 1
Deer, 1
Car, 1
Bear, 1
Bear, 1
Bear, 1
Car, 1
Car, 1
Car, 1
Deer, 1
Deer, 1
River, 1
River, 1
Bear, 2
Car, 3
Deer, 2
River, 2
Bear, 2
Car, 3
Deer, 2
River, 2

Chap.2. Les base des données non relationnelles

2.1. Introduction

Définition: Le NoSQL regroupe de nombreuses bases de données, récentes pour la plupart, qui se caractérisent par une logique de représentation de données non relationnelle et qui n'offrent donc pas une interface de requêtes en SQL.

NoSQL signifie Not Only SQL et non pas No SQL, il s'agit de compléments aux SGBDR pour des besoins spécifiques et non de solutions de remplacement.

Exemple:

BD orientée clé-valeur, BD orientée graphe ,BD orientée colonne, BD orientée document

Le terme NoSQL est pour la première fois propose par **Carlo Strozzi en 1998**, lors de la présentation de son SGBDs relationnels open source. Il l'a appelé ainsi a cause de l'absence d'interface SQL pour interagir avec les bases de données. Les bases NoSQL peuvent bien entendu être utilisées avec du SQL, c'est pourquoi Strozzi précise qu'il est plus pertinent d'utiliser le terme « NoREL » car ces dernières s'abstraient du cote relationnel des données.

Le mot réapparu en 2009 lorsqu'Eric Evans l'utilisa pour caractériser le nombre grandissant de bases de données non-relationnelles lors d'une présentation sur les bases de données distribuées open source.

En 2019, Dans les conférences aux USA ou les notions de BD no SQL ont été discutées.

Les bases des données NoSQL, doivent réponde aux besoins et plusieurs contraintes doivent être prises en compte :

- Les bases des données doivent pouvoir accueillir des données sans forte rigidité, au niveau de leurs structures ;
- Les bases des données doivent pouvoir gérer de très grandes quantités des données
- Les bases des données doivent pouvoir manipuler des types particuliers des données

Les caractéristiques du BD non relationnelle

- Flexibilité de la structure des données: Les BDNR ne sont pas soumises aux contraintes de la normalisation des données et de la définition de clés primaires et étrangères.
- Scalabilité : BDNR sont généralement plus faciles à scaler horizontalement, c'est-à-dire en ajoutant de nouveaux serveurs pour gérer une charge croissante, plutôt que verticalement, c'est-à-dire en ajoutant plus de puissance à un serveur existant. (la capacité d'un produit a s'adapter `a un changement d'ordre de grandeur de la demande, en particulier sa capacité `a maintenir ses fonctionnalités et ses performances en cas de forte demande)
- Requêtes NoSQL : Les BDNR utilisent des requêtes NoSQL qui sont différentes des requêtes SQL utilisées pour les BDR. Les requêtes NoSQL sont souvent plus simples et directes, mais moins expressives que les requêtes SQL.

Exemple: L'équivalent du CREATE TABLE en SQL n'est soit pas nécessaire, soit même pas possible ; on peut directement faire l'équivalent de INSERT INTO.

Cela apporte de la souplesse et de la rapidité, mais se paye avec moins de contrôle et donc de cohérence des données.

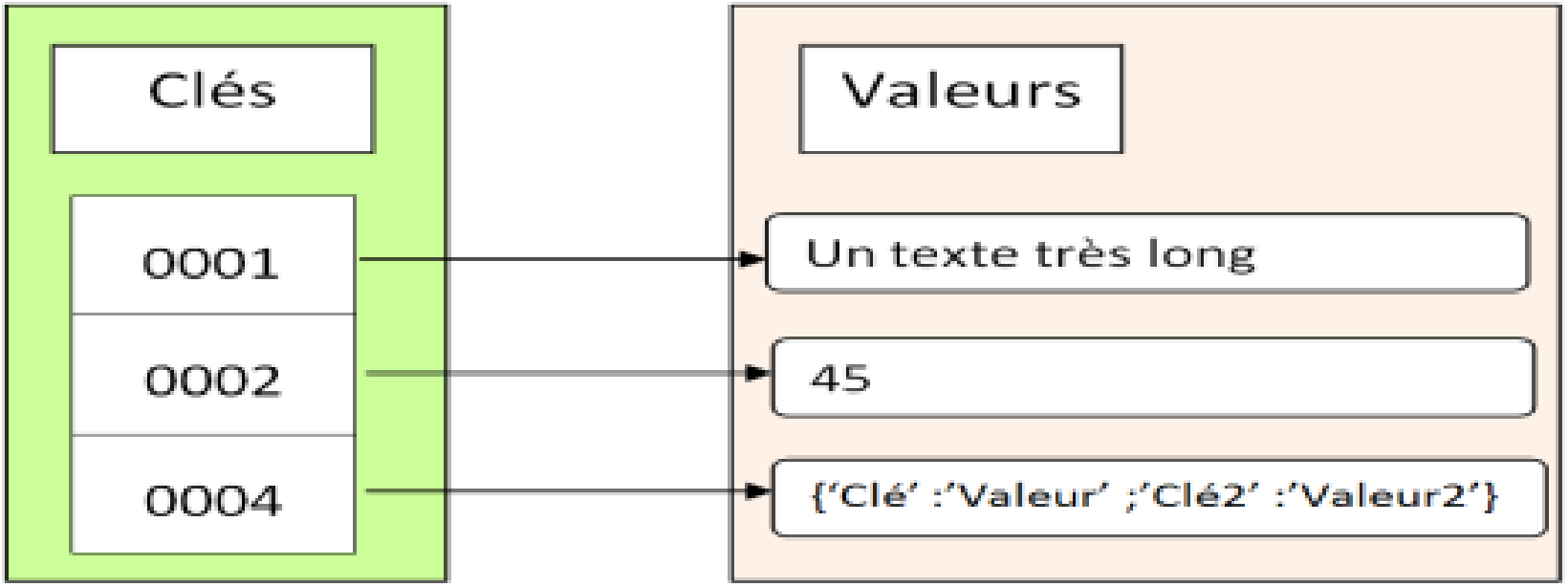
2.2. Les types des bases NoSQL

Dans la mouvance NoSQL, il existe une diversité importante d'approches que nous classons en quatre grandes catégories :

1. Les bases orientées cle-valeur.
2. Les bases orientées colonnes.
3. Les bases orientées documents.
4. Les bases orientées graphes.

2.2.1. Les bases de données orientées clé-valeur

Ce type est probablement le plus connu et le plus basique que comporte la mouvance NoSQL. Son principe est extrêmement simple : chaque objet (valeur) est identifie par une clé unique. Celle-ci représente la seule manière de solliciter l'objet.



Les bases NoSQL orientées clé/valeur

Les bases orientées clé/valeur sont utilisées pour le **stockage temporaire de données**.

Chaque donnée **est référencée par une clé** : c'est à partir de cette dernière que l'on y accède.

Contrairement aux autres bases NoSQL, celles-ci sont plus faciles à utiliser mais leurs usages sont limitées.

- → Chaque valeur peut être de n'importe quel type (chaîne de caractères, nombre, JSON...).
- → Seules les 4 opérations CRUD sont possibles.

⚠ Il n'y a aucune structure sur les valeurs associées aux clés : il faut être prudent avec ces bases.

Les bases NoSQL orientées clé/valeur

L'utilisation des bases NoSQL orientées clé/valeur est très facile, car les clés fournissent une indexation naturelle.

Clé	Valeur
1	https://adresseweb.com
2	356
3	**mail**: monmail@gmail.com **date**: 25/10/2020 13:42:12

⚠ Il n'est pas possible d'effectuer des requêtes sur ces bases.

La structure de l'objet est libre et le plus souvent laisse `a la charge du développeur de l'application (XML, JSON, ...), la base ne gérant généralement que des chaınes d'octets. A fin de communiquer avec cette base, on se résumé généralement qu'a quatre opérations Create Read Update Delete (CRUD).

- **Create** : créer un nouvel objet avec sa clé -» create(key, value).
- **Read** : lit un objet à partir de sa clé -» read(key).
- **Update** : met à jour la valeur d'un objet à partir de sa clé -» update(key, value).
- **Delete** : supprime un objet à partir de sa clé -» delete(key).

La plupart des bases de données de type clé/valeur disposent d'une interface HTTP REST qui permet de procéder très facilement `a des requêtes depuis n'importe quel langage de développement. Ces systèmes affichent des performances exceptionnellement élevées en lecture et en écriture, ainsi qu'une scalabilité horizontale 11 étendue, cela vient du fait que ces types de bases sont réduits `a un simple accès disque.

Les bases NoSQL orientées clé/valeur

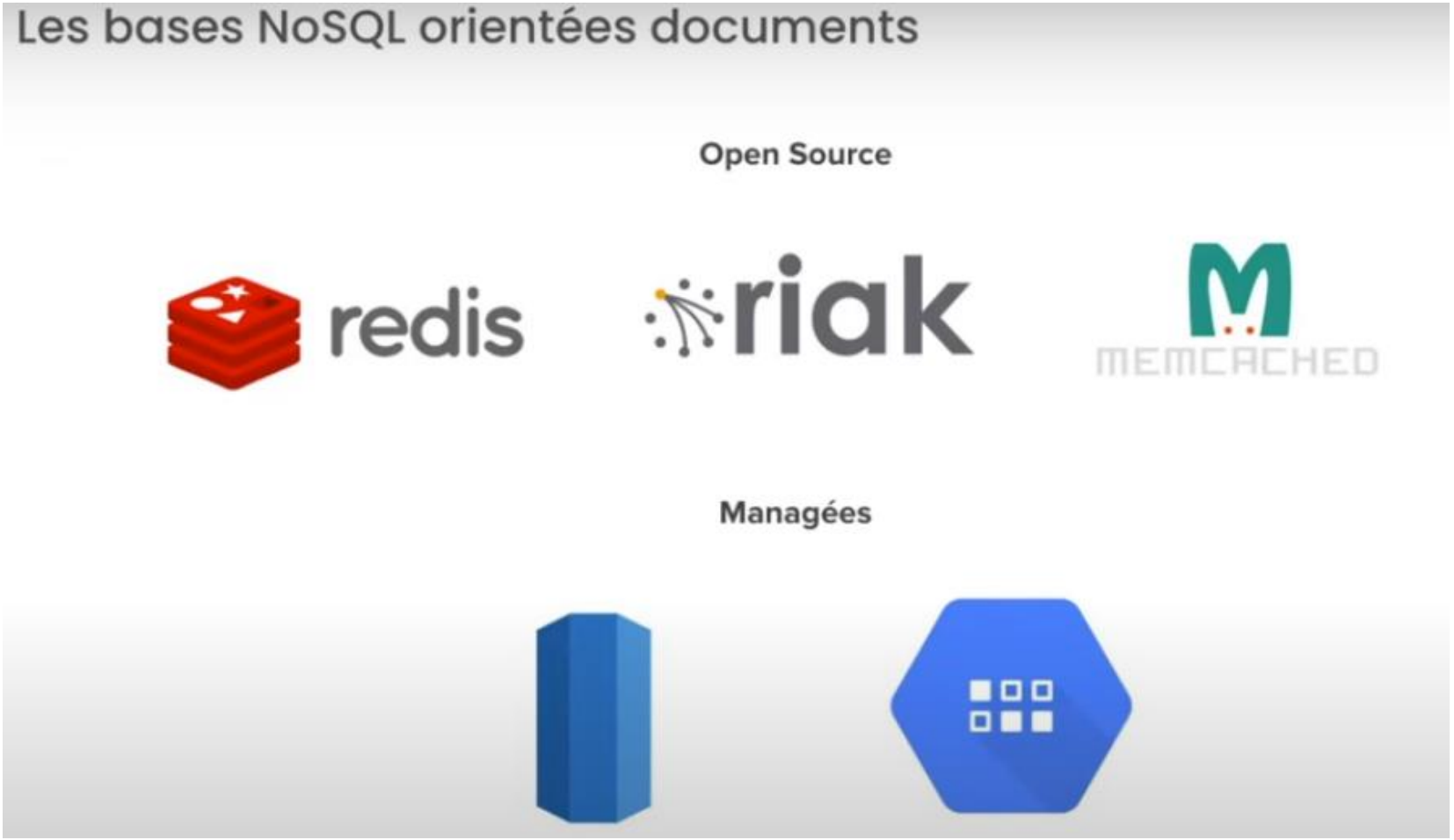
? Quand utiliser un base NoSQL orientée clé/valeur ?

La principale utilisation des bases NoSQL orientées clé/valeur concerne **la mise en cache d'informations**.

- → Panier d'utilisateur (E-Commerce)
- → Collecte d'événements (tracking utilisateur)
- → Partage d'information pour des applications *load-balanced*

Ces bases sont en général plutôt destinées aux développeurs qui mettent en place des solutions de haute disponibilité.

Les outils de Bd no SQL orientée clé/ Valeur



2.2.2 Les bases de données orientées colonnes

Ce modèle ressemble `a première vue `a une table dans un SGBD relationnel `a la différence que le nombre de colonnes est dynamique. En effet, le nombre de colonnes peut varier d'un enregistrement `a un autre ce qui évite de retrouver des colonnes ayant des valeurs NULL.

Les bases NoSQL orientées colonnes

Les bases orientées colonnes peuvent être vues comme **une extension des tables relationnelles**. On y retrouve le principe des **tables** et des **colonnes**.

La principale différence est que **les colonnes ne sont pas fixes pour chaque ligne**.

- → Cela va **libérer de l'espace** de stockage.
- → Les jointures entre les tables **ne sont plus une nécessité**.
- → L'historisation des données peut être **effectuée sur une la ligne**.

Les bases NoSQL orientées colonnes

Cette flexibilité au niveau des colonnes permet également **d'historiser beaucoup plus facilement**.

Product ID	Name	Price 1	Price 2	Price 3
1	Liquide vaisselle	**date**: 01/02/2020 **price**: 2.42€	**date**: 12/05/2020 **price**: 2.48€	
2	Shampooing	**date**: 08/05/2020 **price**: 1.56€	**date**: 12/09/2020 **price**: 1.12€	**date**: 19/09/2020 **price**: 1.56€
3	Fromage blanc	**date**: 12/05/2020 **price**: 2.02€		

Les bases NoSQL orientées colonnes

? Quand utiliser un base NoSQL orientée colonnes ?

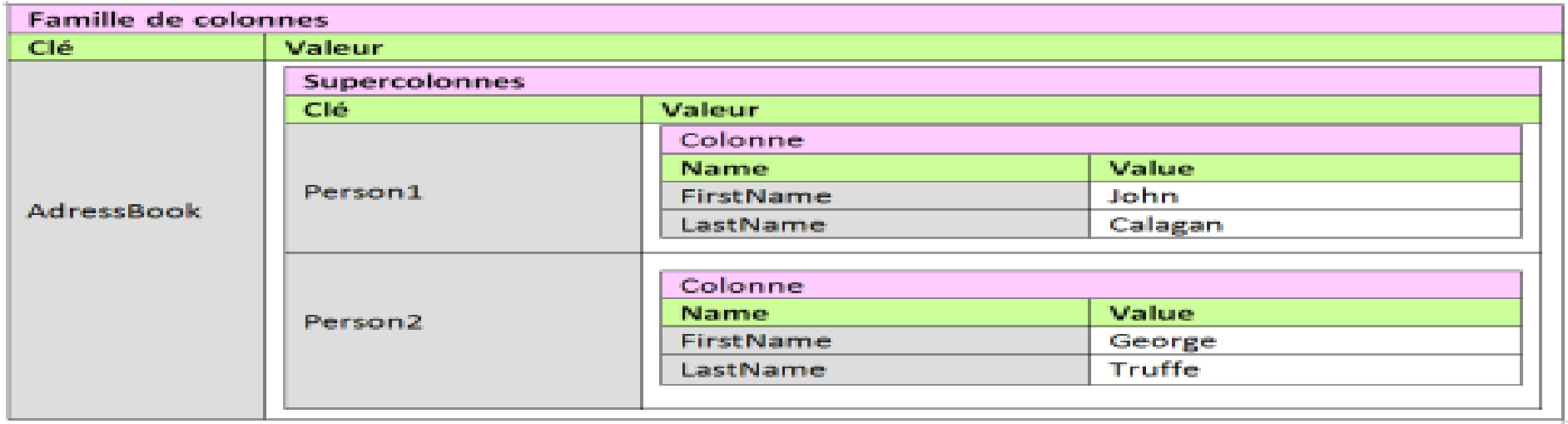
Elles sont adaptées lorsque les données sont très volumineuses et que de nombreux événements surviennent.

- → Analyse de données collectées par webscraping
- → Comportement des utilisateurs
- → Gestion de catalogues (produits, achats, ventes)

Elles gardent, dans leur forme, un certain lien avec les bases relationnelles.

Bien que l'on obtienne au final un schéma relativement proche des bases documentaires, l'organisation sous-jacente des données permet `a ces bases d'exceller dans les traitements d'analyse de données et dans les traitements massifs, notamment via des op´erations de type Map-Reduce.

Bien que l'on trouve des variations en fonction de la base considérée, les concepts essentiels sont les suivant



Les outils utilises pour la BDnoSQL orientée colonnes



- Implémentations les plus connues :

HBase (Open Source de BigTable de Google utilisé pour l'indexation des pages web, Google Earth, Google analytics, ...)

Cassandra (fondation Apache qui respecte l’architecture distribuée de Dynamo d’Amazon, projet né de chez Facebook)

SimpleDB de Amazon.

2.2.3 Les bases de données orientées document

Les systèmes de type documentaire sont composes de collections de documents. La représentation en document est une sorte d'extension du concept clé/valeur, qui se compose de champs et des valeurs associées. Ces dernières peuvent être, soit d'un type simple(entier, chaine de caractère, date, ...), soit elles-mêmes composées de plusieurs couples clé/valeur.

Les bases NoSQL orientées documents

Dans les bases orientées documents, on considère des **collections de documents**, où chaque document contient une liste de champs clé/valeur.

Le format utilisé dans les documents est principalement le JSON ou le XML.

Document (id: 5baf47)

```
{
 "nom": "Liquide vaisselle",
 "images": [
   "https://...",
   "https://..."
 ],
 "specs": {
   "parfum": "Orange"
 }
}
```

Document (id: ea53aa)

```
{
 "nom": "Shampooing",
 "images": [
   "https://...",
   "https://..."
 ],
 "specs": {
   "parfum": "Vanille"
 }
}
```

Document (id: d710bb)

```
{
 "nom": "Fromage blanc",
 "images": [
   "https://...",
   "https://..."
 ],
 "specs": {
   "mat_grasses": "0%"
 }
}
```

JSON (JavaScript Object Notation).
Un fichier au format JSON a pour extension ".json".

Exemple du BDNR orientée document

Rappels sur JSON

JSON : JavaScript Object Notation

- En pratique, aucun lien spécifique avec JavaScript

Format textuel peu typé et moins verbeux que XML

- 4 types de données
  - chaîne, nombre, booléen, null
- 2 structures
  - Objet : ensemble de propriétés
    - Propriété = clé-valeur
    - Clé : chaîne
    - Valeur : un élément d'un type de base, un tableau ou un objet
    - Propriétés séparées par des virgules et encadrées par des accolades
  - Tableaux définis par des crochets et éléments séparés par des virgules
    - Élément : type de base, objet ou tableau

```
{
  "nom": "Michu",
  "prenom": "Robert",
  "age": 70,
  "marie": false,
  "adresse": {
    "rue": "12 rue de Siam",
    "ville": "Brest",
    "codePostal": 29200
  },
  "emploi": null,
  "hobbies": [
    "pétanque",
    "belote",
    "pêche"
  ]
}
```

Collection VENTES :

OID	
5d5gj6ksrg8b45	"novente" : 1 "date" : "04/02/21" "livres" : ["noLivre" : 392649 "noLivre" : 870468]
8gd5gty6u6a34	"novente" : 2 "date" : "04/02/21" "livres" : ["noLivre" : 486734]

Collection LIVRES :

OID	

8jfr56gh5837h7f	"nolivre" : 392649
	"titre" : "Bases de données NoSQL"
	"auteur" : {
	"nom" : Dupont
	"prenom" : Jean
	}

3j67hyr67De67	"nolivre" : 870468
	"titre" : "Hadoop facile"
	"auteur" : {
	"nom" : Martin
	"prenom" : Anne
	}

8hjt7hyr67fg67	"nolivre" : 486734
	"titre" : "Usage de MapReduce"
	"auteur" : {
	"nom" : Schmit
	"prenom" : Paul
	}

Exemple de collection sportif


```
[
    {
        "_id": 1,
        "prenom": "Roger",
        "nom": "Blanchard",
        "age": 55,
        "genre": "homme",
        "adresse": {
            "rue": "12 rue de Siam",
            "ville": "Brest",
            "codePostal": 29200
        },
        "disciplines": [
            "100m",
            "200m",
            "descente"
        ]
    },
    {
        "_id": 2,
        "prenom": "Simone",
        "nom": "Blanchard",
        "age": 52,
        "genre": "femme",
        "adresse": {
            "rue": "12 rue de Siam",
            "ville": "Brest",
            "codePostal": 29200
        },
        "disciplines": [
            "200m 4 nages",
            "100m libre"
        ]
    },
    {
        "_id": 3,
        "prenom": "Gérard",
        "nom": "Lebreton",
        "age": 24,
        "genre": "homme",
        "adresse": {
            "rue": "20 rue Saint-Michel",
            "ville": "Rennes",
            "codePostal": 35000
        },
        "disciplines": [
            "marathon",
            "slalom",
            "géant",
            "100m libre"
        ]
    },
    {
        "_id": 4,
        "prenom": "Régine",
        "nom": "Mouvier",
        "age": 31,
        "genre": "femme",
        "adresse": {
            "rue": "153 rue Jean Jaures",
            "ville": "Brest",
            "codePostal": 29200
        },
        "disciplines": [
            "descente",
            "géant",
            "100m libre"
        ]
    }
]
```

Les bases NoSQL orientées documents

? Quand utiliser un base NoSQL orientée document ?

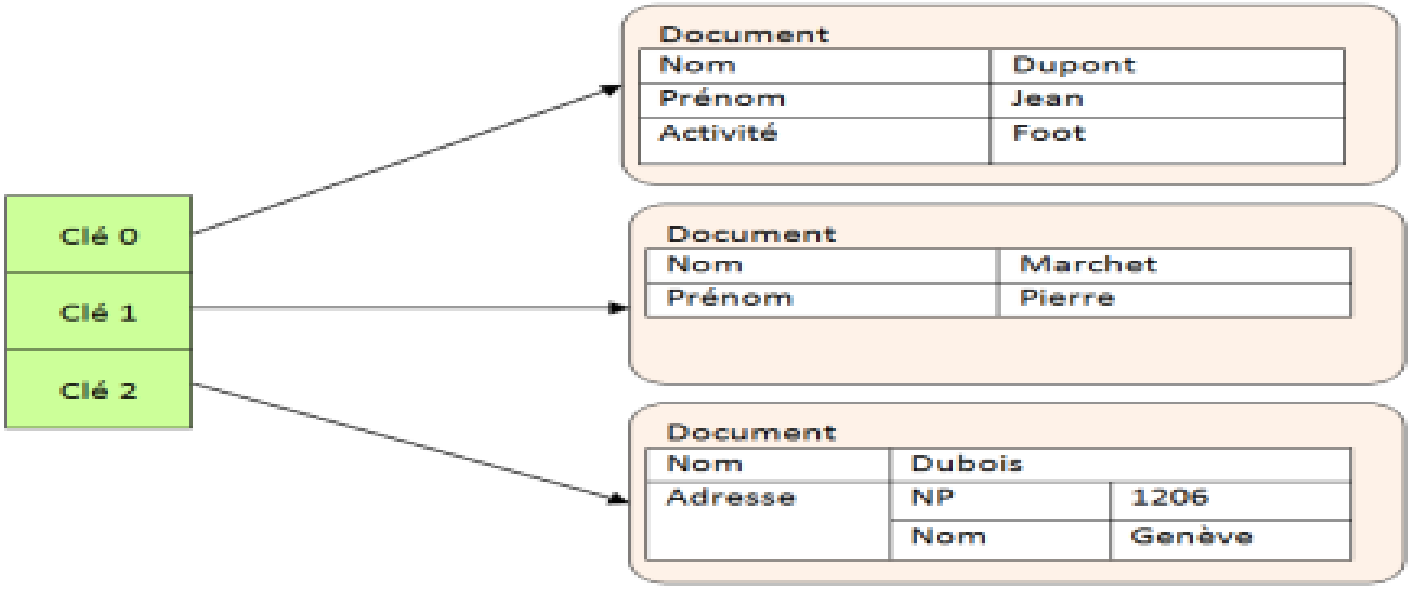
Il faut les utiliser lorsqu'il n'y **pas de relations entre les documents et les collections**.

- → Gestion de catalogues diversifié (produits, achats, ventes)
- → Base d'utilisateurs et de clients
- → Web Analytics

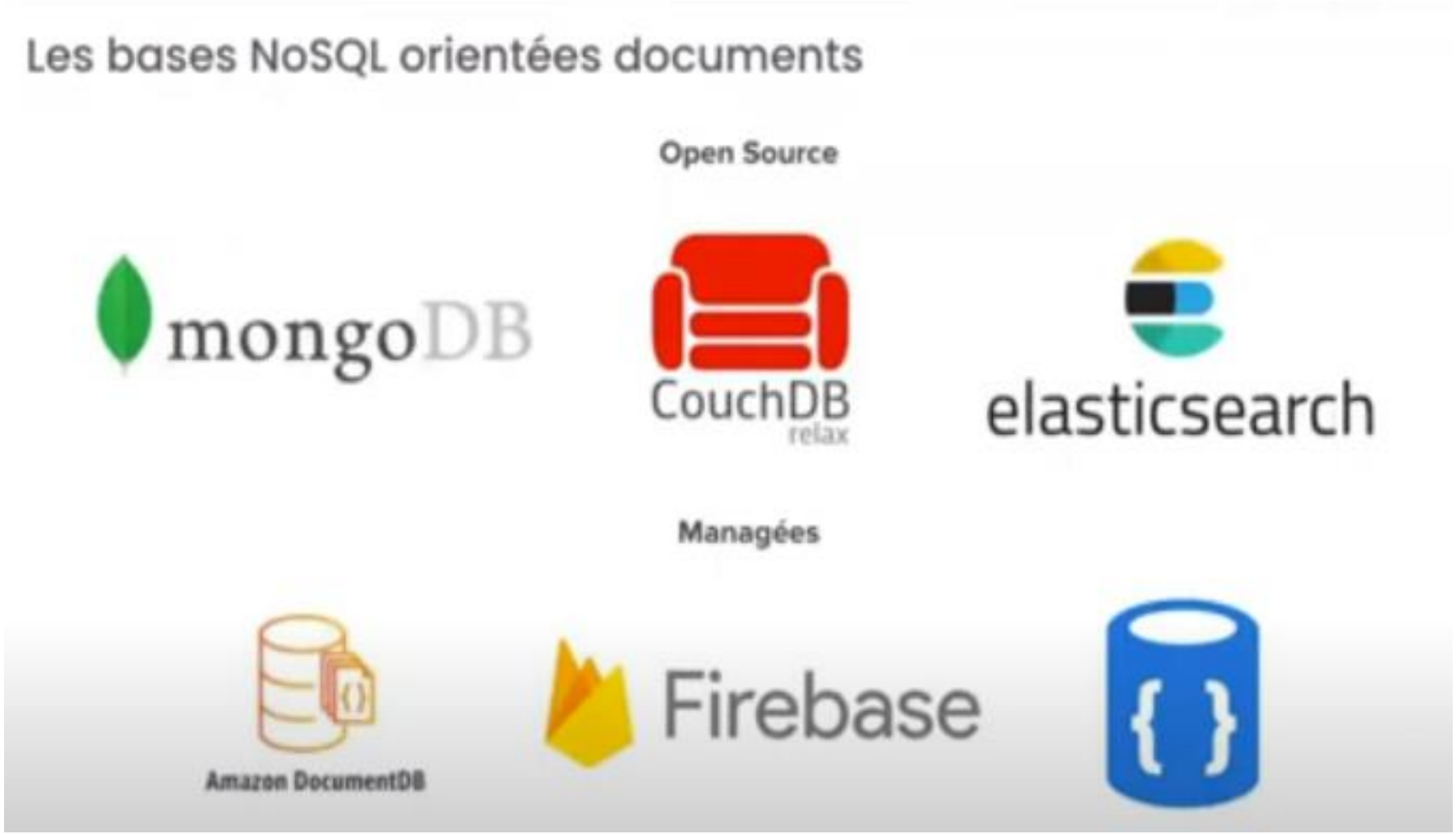
S'il y a une cohérence assez forte entre les données (colonnes plus ou moins similaires), on préfèrera utiliser une base NoSQL orientée colonnes.

- Aucune structure au niveau des documents, c est une collection des documents malgré leur différence.
- Elle utilisée quand , in a aucune rigidité des données regroupées dans les documents.
- Les deux types de base noSQL sont les plus utilises

Par ailleurs, les documents sont organises de manière hiérarchique `a l'image de ce que permettent les fichiers de type XML ou JSON, elles sont dites Schemaless ou en français sans schéma défini ce qui signifie : il n'est pas nécessaire de définir au préalable les champs dans le document[B6]. De plus, le stockage structure des documents leur confère des fonctionnalités simples dont la plus ´évidente est la capacité `a effectuer des requêtes sur le contenu des objets.

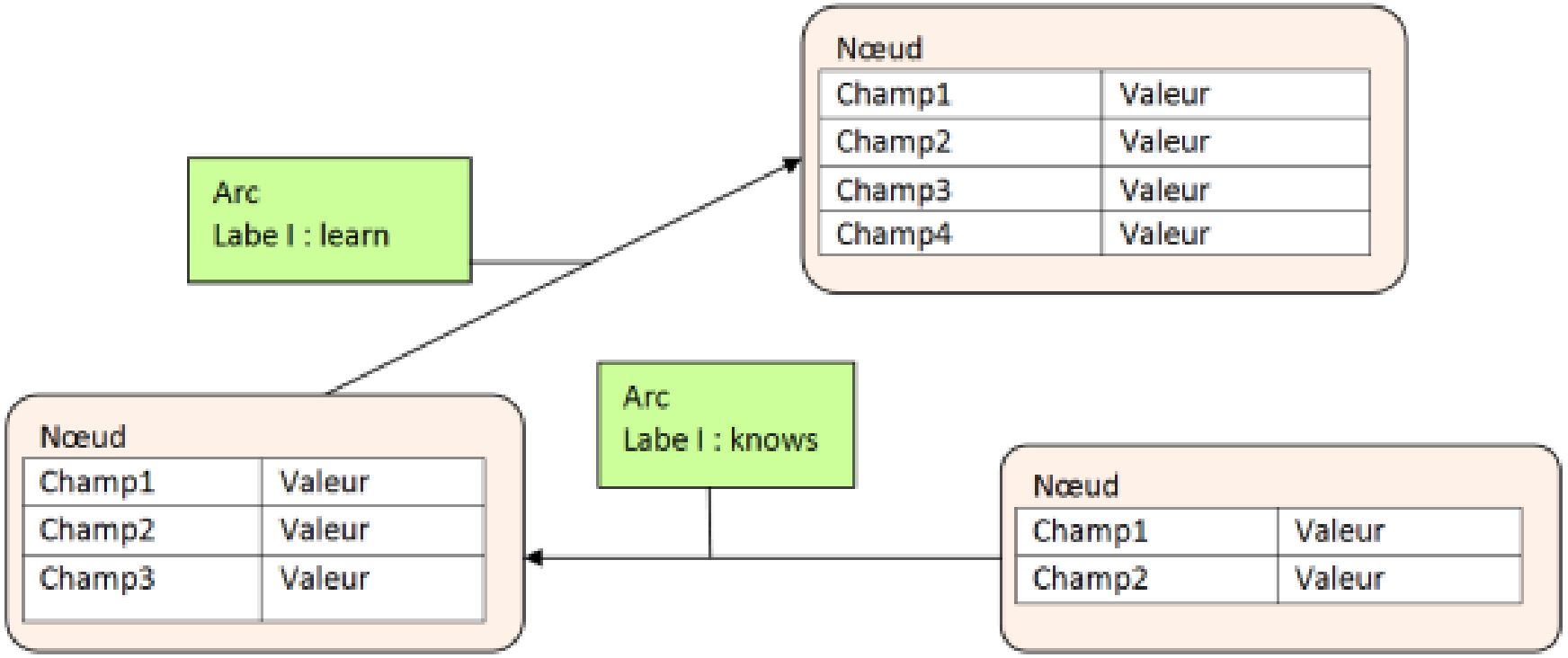


Les outils utilises pour la BDnoSQL orientée documents



2.2.4. Les bases de données orientées graphes

Les bases orientées graphe sont les moins connues de la mouvance NoSQL. Ces bases permettent la modélisation, le stockage ainsi que le traitement de données complexes reliées par des relations. Elle se compose de deux concepts ; un moteur de stockage pour les objets qui est une base documentaire ou chaque entité de cette base est nomme nœud, et un mécanisme qui decrit les arcs : c'est les relations entre les objets, elles contiennent des propriétés de type simple (integer, string, date..)


Nœud
Champ1
Valeur
Champ2
Valeur
Champ3
Valeur
Champ4
Valeur
Arc
Labe l : learn
Arc
Labe l : knows
Nœud
Champ1
Valeur
Champ2
Valeur
Champ3
Valeur
Nœud
Champ1
Valeur
Champ2
Valeur

Les autres outils

Les autres bases NoSQL

Bases orientées graphes



Bases orientées séries temporelles



Evolution

Neo4j est un système de gestion de base de données orienté graphe, développé en Java par la société Neo4j. Il permet de représenter les données sous forme de nœuds et de relations, offrant une modélisation flexible et proche des concepts métier.

Neo4j est une base de données graphique qui permet de gérer et d'analyser des données interconnectées sous forme de graphes. Contrairement aux bases de données relationnelles classiques, elle représente les informations à travers des nœuds (entités), des relations (connexions entre les nœuds) et des propriétés (informations sur les nœuds et relations). Cela permet d'exécuter des requêtes complexes sur des relations profondes et d'analyser rapidement les connexions entre différentes entités. Utilisée dans des domaines comme les réseaux sociaux, la détection de fraudes, et les systèmes de recommandation, Neo4j est particulièrement performante pour les applications nécessitant l'exploration de relations complexes. Grâce à son langage de requête Cypher, Neo4j facilite l’interrogation des graphes et offre une grande flexibilité dans la gestion des données

Neo4j vise à fournir une solution efficace pour le stockage et
la gestion de données fortement connectées, où les relations entre les données sont primordiales.

Il est particulièrement adapté aux applications nécessitant des parcours complexes,

comme les réseaux sociaux, la détection de fraudes ou les systèmes de recommandation.

Objectifs clés de Neo4j

1. **Représenter des relations complexes :**

Neo4j permet de modéliser facilement des relations entre entités, ce qui est particulièrement utile dans des domaines comme les réseaux sociaux, les recommandations de produits, l'analyse de chaînes logistiques, ou la détection de fraudes.

2**. Optimisation des requêtes sur les graphes :** Contrairement aux bases de données relationnelles qui peuvent devenir lentes lors de l'exécution de requêtes complexes sur des relations profondes, Neo4j est optimisé pour effectuer des parcours de graphes rapides, ce qui permet de naviguer efficacement à travers des réseaux d'éléments interconnectés.

3. **Gestion de données fortement connectées :** L'un des principaux avantages de Neo4j est de gérer des jeux de données où les relations entre les données sont aussi importantes que les données elles-mêmes. Cela le rend idéal pour les applications comme l'analyse de réseaux sociaux, les moteurs de recommandations, la gestion des connaissances, etc.

4. **Simplicité d'utilisation :** Grâce à son langage de requête Cypher, Neo4j permet aux utilisateurs de poser des questions sur les graphes de manière intuitive, facilitant la compréhension et l'exécution de requêtes complexes sur des données interconnectées.

5. **Scalabilité et flexibilité :** Neo4j offre des solutions pour évoluer en fonction des besoins de l'entreprise, tout en restant flexible pour modéliser une variété de types de relations et de données. Il est conçu pour travailler avec des graphes de grande taille, offrant des options pour le déploiement sur des infrastructures distribuées.

Exemples des Requettes

Création de nœuds:

```
CREATE (:Person {name: 'Alice', age: 30})
```

Création de relations:

```
MATCH (a:Person {name: 'Alice'}), (b:Person {name: 'Bob'})
CREATE (a)-[:FRIEND]->(b)
```

Recherche de nœuds:

```
MATCH (p:Person) RETURN p
```

Recherche avec condition:

```
MATCH (p:Person {name: 'Alice'}) RETURN p
```

Mise à jour des propriétés:

```
MATCH (p:Person {name: 'Alice'})
SET p.age = 31
```

Suppression de relations:

```
MATCH (a:Person {name: 'Alice'})-[r:FRIEND]->(b:Person {name: 'Bob'})
DELETE r
```

Création d'un index:

```
CREATE INDEX FOR (p:Person) ON (p.name)
```

Requête avec tri:

```
MATCH (p:Person)  RETURN p.name, p.age ORDER BY p.age DESC
```

Compter les nœuds:

```
MATCH (p:Person) RETURN count(p)
```

Neo4j se distingue comme une base de données graphique puissante et flexible, spécialement

conçue pour gérer des données interconnectées et effectuer des analyses complexes sur les relations entre les entités. Grâce à son modèle de données basé sur les graphes et son langage de requête Cypher, Neo4j offre des performances exceptionnelles pour des cas

d'utilisation tels que l'analyse de réseaux sociaux, les systèmes de recommandation, et la détection de fraudes.

Son efficacité pour explorer des relations profondes et complexes le rend idéal pour des applications nécessitant une compréhension approfondie des connexions entre les données.

Avec sa scalabilité et sa capacité à évoluer, Neo4j est un choix incontournable pour les entreprises et développeurs qui cherchent à exploiter la puissance des graphes

pour résoudre des problèmes de données complexes

Présentation de Firebase

Firebase est une plateforme de développement d'applications qui fournit une gamme complète d'outils pour aider les développeurs à créer des applications performantes et évolutives. Racheté par Google en 2014, Firebase s'est imposé comme une solution incontournable, notamment grâce à ses services de **base de données, d'authentification, d'hébergement, de notifications push et d'analyse des performances**.

Parmi ses principaux composants, on retrouve :

- **Firebase Authentication** : un service d'authentification permettant de gérer les utilisateurs via des comptes Google, Facebook, Twitter, email, etc.
- **Cloud Firestore et Firebase Realtime Database** : des solutions de base de données NoSQL pour stocker et synchroniser des données.
- **Firebase Hosting** : un service d'hébergement pour les applications web.
- **Firebase Cloud Functions** : une plateforme serverless pour exécuter du code backend en réponse à des événements déclenchés par Firebase.
- **Firebase Cloud Messaging (FCM)** : un service permettant l'envoi de notifications push sur Android, iOS et le web.

Présentation de Firebase

Dans son livre *Building Applications with Firebase*, Raj V. Patel explique :

« Firebase n'est pas simplement une base de données, c'est un écosystème complet qui réduit la complexité du développement d’applications en fournissant une infrastructure entièrement managée. »

Firebase est particulièrement apprécié pour sa simplicité de mise en œuvre et son intégration native avec l’écosystème Google et Android.

Cloud Firestore : Une Base de Données NoSQL Puissante

Firestore est l'une des bases de données proposées par Firebase, conçue pour stocker des **données semi-structurées sous forme de documents et de collections**. Contrairement à une base SQL traditionnelle, Firestore ne fonctionne pas avec des tables et des relations, mais avec un modèle orienté-documents similaire à **MongoDB**.

Fonctionnement de Firestore

Dans *Cloud Computing: Principles and Paradigms*, Rajkumar Buyya explique l'intérêt des bases de données cloud comme Firestore :

« La gestion distribuée des données dans le cloud permet une haute disponibilité et une récupération rapide en cas de panne, ce qui rend les solutions NoSQL idéales pour les applications modernes. » Voici les axes sur lesquels tournent son fonctionnement:

- **Structure de données** : Firestore organise les données en **collections** contenant des **documents**. Chaque document est un ensemble de paires clé-valeur et peut contenir des sous-collections.
- **Requêtes flexibles** : Firestore prend en charge des requêtes complexes avec filtrage et tri avancés.
- **Mode hors ligne** : Les données peuvent être mises en cache localement et synchronisées avec le cloud dès qu'une connexion est rétablie.
- **Intégration avec Firebase** : Firestore est nativement intégré avec d'autres services Firebase comme **Authentication, Cloud Functions et Analytics**.
- **Sécurité** : Les règles de sécurité Firestore permettent de définir des accès précis pour chaque utilisateur ou groupe d'utilisateurs.

Points forts de Firestore

Martin Kleppmann insiste sur l'importance de la scalabilité dans les bases NoSQL comme Firestore : « Une base de données bien conçue doit être capable de gérer la montée en charge sans compromettre la latence ou la disponibilité. » Ceci montre certains points forts:

- **Synchronisation en temps réel** : Firestore met à jour les données instantanément sur tous les appareils connectés.
- **Scalabilité automatique** : Il est conçu pour gérer des millions d'utilisateurs sans intervention humaine.
- **Haute disponibilité et fiabilité** : Firestore est hébergé sur l'infrastructure Google Cloud, garantissant une haute disponibilité.
- **Facilité d'utilisation** : Son API est simple et bien documentée, avec des SDK disponibles pour plusieurs plateformes (Android, iOS, Web).
- **Sécurité avancée** : Les règles de sécurité permettent un contrôle fin sur les accès et modifications des données.
- **Mode hors ligne** : Les applications continuent de fonctionner même sans connexion Internet, ce qui est un avantage majeur pour les applications mobiles.

Limites de Firestore

Dans *Database Internals: A Deep Dive into How Distributed Data Systems Work*, Alex Petrov met en garde contre les limites des bases NoSQL : « Bien que les bases NoSQL offrent une grande flexibilité, elles exigent une réflexion approfondie sur la structuration des données pour éviter les inefficacités et les coûts cachés. » Voici certaines limites:

- ✓ **Coût potentiellement élevé** : Firestore facture en fonction du nombre de lectures, écritures et suppressions de documents, ce qui peut devenir coûteux pour les applications ayant un fort volume de requêtes.
- ✓ **Limitations des requêtes** : Contrairement aux bases SQL, Firestore ne permet pas les **jointures complexes**, ce qui peut nécessiter une structuration différente des données.
- ✓ **Latence accrue par rapport aux bases SQL** : Bien que Firestore soit performant, certaines requêtes peuvent être plus lentes comparées aux bases de données relationnelles optimisées.
- ✓ **Règles de sécurité complexes** : Une mauvaise configuration des règles de sécurité peut compromettre l'intégrité des données ou restreindre excessivement l'accès aux utilisateurs.
- ✓ **Écriture limitée par transaction** : Firestore impose une limite sur le nombre d'opérations effectuées dans une transaction, ce qui peut poser problème pour certaines applications nécessitant des mises à jour complexes.

Comparaison avec d'autres solutions

Fonctionnalité	Firestore	Realtime Database	MongoDB	MySQL/PostgreSQL
Modèle de données	Documents et collections	JSON hiérarchique	Documents et collections	Tables et relations
Requêtes complexes	Oui, mais limitées	Non	Oui	Oui
Synchronisation en temps réel	Oui	Oui	Non	Non
Scalabilité	Excellente	Moyenne	Excellente	Moyenne
Mode hors ligne	Oui	Partiel	Oui	Non
Coût	Facturation par opération	Facturation par volume	Coût flexible	Dépend du serveur

Conclusion

Firebase est une plateforme robuste et bien intégrée qui simplifie grandement le développement d'applications, notamment grâce à Firestore, sa base de données NoSQL moderne. Avec sa synchronisation en temps réel, son évolutivité et son intégration avec l'écosystème Google, Firestore représente une solution puissante pour les applications nécessitant des interactions rapides et fluides entre les utilisateurs.

Cependant, Firestore n'est pas exempt de limites. Son coût basé sur le nombre de requêtes peut devenir un facteur contraignant pour les grandes applications, et son absence de relations entre les collections peut compliquer certaines structures de données.

Comme l’affirme Michael Stonebraker dans *The End of an Architectural Era (It’s Time for a Complete Rewrite)* : « Aucune base de données ne peut répondre à tous les besoins. Le choix d’un système doit être dicté par les contraintes spécifiques du projet et les objectifs à long terme. »

En définitive, Firestore convient parfaitement aux applications qui nécessitent une synchronisation en temps réel, une scalabilité automatique et un déploiement rapide. Mais pour des besoins plus complexes impliquant des relations de données avancées et des coûts optimisés, une base de données relationnelle traditionnelle pourrait être un meilleur choix.

Les inconvénients

Les avantages apportes par les bases de données NoSQL ne sont pas sans contreparties. En effet comme toute avancée technologique les bases de données NoSQL ont aussi des inconvénients, on en citera :

La maturité : Les SGBD relationnels sont la depuis plus de 30 ans, ainsi, cette pérennité est un signe de **réconfort pour les responsables** vu qu'ils sont de nature **stable et riche** en fonctionnalités. En comparaison, la plupart des SGBD NoSQL sont en pre-production et ont encore beaucoup de fonctionnalités futures `a implémenter.

Fonctionnalités réduites : Les bases de données NoSQL sont principalement conçues pour stocker des données et offrent très peu de fonctionnalités autres que celle-la.

Lorsque les transactions entrent dans l'equation, les bases de données relationnelles restent le meilleur choix. Voila pourquoi les bases NoSQL ne pourront jamais remplacer entièrement SQL.

L'administration : Un des buts des bases de données NoSQL était d'en simplifier l'administration, mais la réalité est différente. Aujourd'hui, les solutions NoSQL demandent beaucoup de compétences pour leur installation ainsi que des efforts conséquents lors de leur maintenance.

L'expertise : On retrouve dans le monde des millions de développeurs, opérant, dans différents businesses, qui sont familiers des concepts ainsi que de la programmation dans un environnement de bases de données relationnelles. Dans le monde NoSQL, presque tous les développeurs sont en apprentissage avec la technologie.

Comparaison entre SGBDR et les bases de données Nosql

• *Les structures de données*	
SGBDR	**NoSQL**
• Le stockage de données est organisé sur le principe de tables reliées entre elles. • La structure et les types des données sont rigides, c'est-à-dire fixés à l'avance avant d'implémenter une logique métier. • Les attributs sont fortement typés, et selon les principes édictés par Codd, la normalisation correcte des structures implique qu'il n'y ait aucune redondance des données. • Les tables SQL imposent un modèle de données strictes, donc il est difficile de faire des erreurs.	• Le noSQL repousse les limites, il est beaucoup plus flexible. • Les données ne sont pas typées ni limitées. • On trouve moins de contraintes. • On est plus proche d'un agrégat que d'une table, parce que la clé revêt une plus grande importance.

• *Le modèle et la logique*	
SGBDR	**NoSQL**
• Pas d'ajout de données tant qu'on ne définitc pas les tables et les types de champs dans ce que l'on appelle un schéma • Le schéma SQL contient d'autres informations : Clés primaires – index, contraintes, fonction, procédures stockées...	• Les données peuvent être ajoutées n'importe où et à tout moment. • Il n'est pas nécessaire de spécifier une conception de document ou même une collection à l'avance.

• *Les jointures*	
SGBDR	**NoSQL**
• Les requêtes SQL offrent une puissante clause JOIN. • Nous pouvons obtenir des données reliées dans plusieurs tables en utilisant une seule instruction SQL.	• NoSQL n'a pas toujours d'équivalent de JOIN.

• *Le partitionnement de données*	
SGBDR	**NoSQL**
• L'un des problèmes de la normalisation dans un SGBDR concerne la distribution des données et du traitement. ***Exemple :*** *Si on stocke des données qui ont un rapport entre elles, comme des clients,des commandes, des factures, des lignes de facture, etc., dans des tables différentes, on rencontrera un problème lorsque du partitionnement ces données. On doit alors s'assurer que les données en rapport les unes avec les autres se trouvent sur le même serveur.*	• La réplication est essentiellement destinée à pallier les pannes en dupliquant les données sur plusieurs serveurs et en permettant donc qu'un serveur prenne la relève quand un autre vient à faillir. • Le fait de disposer des mêmes données sur plusieurs serveurs par réplication ouvre également la voie à la distribution de la charge (en recherche, en insertion) et donc à la scalabilité.

- ***L'integrité des données***

SGBDR	NoSQL
• La plupart des bases de données SQL nous permettent d'appliquer des règles d'intégrité de données à l'aide de contraintes de clés étrangères. • Elles peuvent valider nos données et nos mises à jour (définition de contraintes), mais elles les rendent du coup très strictes.	• Ces options d'intégrité ne sont pas disponibles dans les bases de données NoSQL. • Elles sont plus flexibles mais ne valident pas les données. • Il en revient au développeur d'ajouter des contrôles dans la logique du programme car la base de données elle-même n'empêchera pas les erreurs. • Vous pouvez stocker ce que vous voulez indépendamment de tout autre document. • Idéalement, un seul document sera la seule source de toutes les informations sur un élément.

- ***Transaction***

SGBDR	NoSQL
• Plusieurs mises à jour peuvent être exécutées au sein d'une même transaction afin de garantir le succès ou l'échec de l'exécution du code SQL.	• La modification d'un document unique est atomique (si vous mettez à jour trois valeurs dans un document, les trois sont mis à jour avec succès ou ils restent inchangés. • Il n'y a pas vraiment d'équivalent de la transaction SQL pour les mises à jour de plusieurs documents. • Il semble alors clair qu'il faille gérer la notion d'atomicité dans le code.

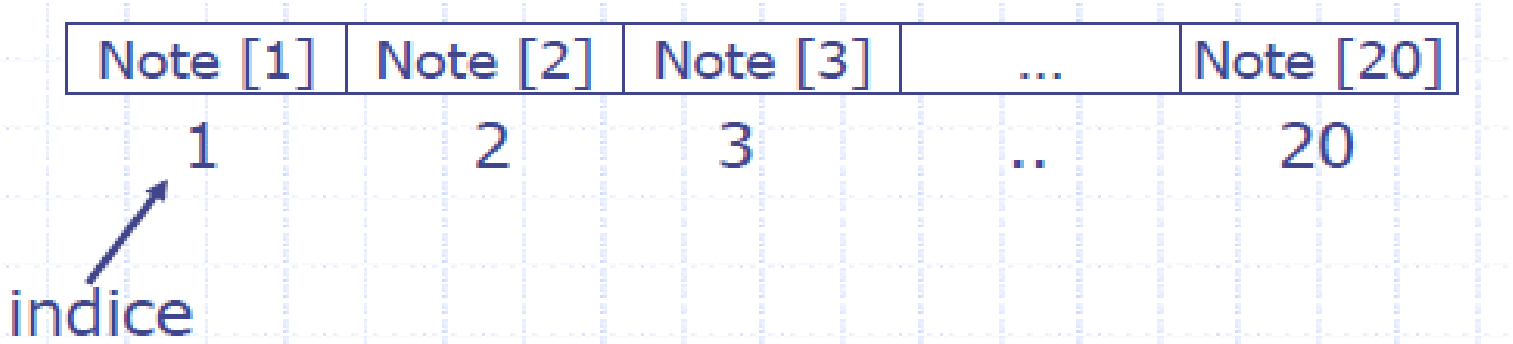
• *Performance*	
SGBDR	**NoSQL**
• Le SQL est moins préformant en revanche, la conception du globale projet et les exigences en matière d'organisation de données seront déterminant avant même le choix du SGBD • Une base de données SQL bien conçue sera certainement supérieure à celle d'un équivalent NoSQL mal conçu et vice versa.	• NoSQL est plus rapide que SQL. • Le principe de « denormalization » induit une représentation plus simple et permet donc de récupérer toutes les informations sur un élément spécifique dans une seule requête, il n'y a donc pas besoin de liens JOIN ou de requêtes SQL complexes. • la redondance des informations alourdie considérablement les opérations de mise à jour.

En conclusion, nous pouvons dire que le NoSQL est une technologie en plein essor qui, en plus de ses performances techniques avérées, constitue un véritable marche qui profite aux entreprises du secteur. Il est important de noter que le NoSQL n'est en aucun cas une négation du SQL, ni un remplaçant du SQL. En ce qui concerne l'avenir du NoSQL nous pouvons affirmer que cette technologie cohabitera avec le SQL qui demeure pour l'heure le langage le plus utilise

Chap.3.Traitement des données avec Python

3.1. Les listes

Une liste possède un identificateur et chaque élément du tableau est repéré par un indice précisant sa position au sein du tableau.



On notera un élément i du tableau T de la manière suivante T[i]

1er élément : T[0]

2ième élément T[1]

3ième élément T[2] etc.

Un indice peut être une expression arithmétique par exemple : NOTE[i+2], NOTE[2*i+j]

Affichage d’une liste en python

```
import os

import sys

liste =[]

n=int(input("entrer la taille de la liste n"))

for i in range(n):

   element=int(input("saisir l element"))

   liste.append(element)

print(liste)
```

```
import os
import sys
liste1 =[]
n=int(input("Saisir la taille du vecteur n = "))
for i in range(n):
   element=int(input("Element du vecteur  = "))
   liste1.append(element)
print(liste1)
print("==================================")
liste2 =[]
n=int(input("Saisir la taille de la deuxieme liste n = "))
for i in range(n):
   element=int(input("Element du vecteur  = "))
   liste2.append(element)
print("==================================")
print("la concatenation de deux liste est")
liste1.extend(liste2)
print(liste1)
```

QUELQUES FONCTIONS POUR LE TRAITEMENT DES LISTES

Le type de données liste possède d'autres méthodes. Voici toutes les méthodes des objets listes :

`append(x)`

Equivalent à `a.insert(len(a), x)`.

`extend(L)`

Rallonge la liste en ajoutant à la fin tous les éléments de la liste donnée ; équivaut à `a[len(a):] = L`.

`insert(i, x)`

Insère un élément à une position donnée. Le premier argument est l'indice de l'élément avant lequel il faut insérer, donc `a.insert(0, x)` insère au début de la liste, et `a.insert(len(a), x)` est équivalent à `a.append(x)`.

`remove(x)`

Enlève le premier élément de la liste dont la valeur est `x`. Il y a erreur si cet élément n'existe pas.

`pop([i])`

Enlève l'élément présent à la position donnée dans la liste, et le renvoie. Si aucun indice n'est spécifié, `a.pop()` renvoie le dernier élément de la liste. L'élément est aussi supprimé de la liste.

`index(x)`

Retourne l'indice dans la liste du premier élément dont la valeur est `x`. Il y a erreur si cet élément n'existe pas.

`count(x)`

Renvoie le nombre de fois que x apparaît dans la liste.

`sort()`

Trie les éléments à l'intérieur de la liste.

`reverse()`

Renverse l'ordre des éléments à l'intérieur de la liste.

Un exemple qui utilise toutes les méthodes des listes :

1. calculer la somme des éléments de la liste
2. Afficher la moyenne des éléments de la liste
3. Afficher le nombre d élément nul , negatif

EXEMPLES

```
import os
import sys
liste =[]
n=int(input("Saisir la taille du vecteur n = "))
for i in range(n):
   element=int(input("Element du vecteur  = "))
   liste.append(element)
print("le premier element est ",liste[0])
print("le dernier element est ",liste[n-1])
print("L element minimal est ",min(liste))
t=min(liste)
q=liste.index(t)
m=max(liste)
l=liste.index(m)
print("l indice de l element minimal",q)
print("LA LISTE TRIEE EST LA SUIVANTE")
liste.sort()
print(liste)
```

3.2.Les matrices sans numpy

```
import os
import sys
row=int(input("Entrer le nombre de ligne "))
col=int(input("Entrer le nombre de colonne "))
a=[]
for i in range(row):
        a.append([0]*col)
        for j in range(col):
                a[i][j]=int(input("Entrez les elements de la
matrice"))
print(a)
```

Exemple 2

```
import os
import sys
row=int(input("Entrer le nombre de ligne "))
col=int(input("Entrer le nombre de colonne "))
a=[]
for i in range(row):
        a.append([0]*col)
        for j in range(col):
                a[i][j]=int(input("Entrez les éléments de la matrice"))
print(a)
k=0
for i in range(row):
    for j in range(col):
        if a[i][j]==0:
            k=k+1
print(k)
```

3.3. Rappel sur le bibliothèque numpy

```
import numpy as np
a = np.array([[1,2],[3,1],[1,4]])
print(a)
// Affichage d un élément de la matrice
print(a[2,2])

print(a.dtype)  #type de données de la matrice
print(a.size), le nombre d élément donc ligne fois colonne
```

Fonctions principales de NumPy

```
import numpy as np
a = np.array([[1,2],[3,1],[1,4]])
print(a)
// Affichage d un élément de la matrice
print(a[2,2])

print(a.dtype)  #type de données de la matrice
print(a.size), le nombre d élément donc ligne fois colonne
```

```
a =np.arange(0,2).reshape(1,2)// Composition de la matrice par les elets connus
print(a)
//Transformation de la liste en matrice
a = np.array([3,4,6,9,5,2])
b = a.reshape(3,2)
print(b)
// ajout des  colonnes
a = np.array([[1,2,5],[3,2,1],[1,1,4]])
d = np.array([[7],[6],[4]])
print(np.append(a,d,axis=1))
```

```
// Redimensionnement
d = np.resize(a,new_shape=(4,3))
print(d)
//Somme des lignes
somm= np.sum(d,axis=0)
print(somm)
//sommesdes colonnes
somm= np.sum(d,axis=1)
print(somm)
// Element maxi  par colonnes
print(np.max(d,axis=1))
// Element maxi  par lignes
print(np.max(d,axis=0))
// Element maxi  par lignes
print(np.max(d,axis=0))
```

```
// Element maxi  par colonnes
print(np.max(d,axis=1))
// Moyenne des colonnes
print(np.mean(d,axis=1))
// Moyenne des lignes
print(np.mean(d,axis=0))
// Transpose de la matrice
print(np.transpose(d))
// Produit de deux matrice
print(np.dot(a,m))
// Calcul du determinent
print(np.linalg.det(m))
//matrice symerique
o = np.dot(np.transpose(t),t)
 print(o)
```

Suppression de la ligne (axis = 0) via son indice (n°1)
print(np.delete(a,1,axis=0))

tri des lignes (axis = 0) pour chaque colonne
print(np.sort(v,axis=0))

somme cumulée des valeurs en ligne pour chaque colonne
print(np.cumsum(v,axis=0))

Inversion
print(np.linalg.inv(y))

Exercices

1. Créez le matrice a
2. Supprimez la 3 eme ligne de la matrice a
3. Supprimez la dernière colonne de la matrice a
3. Triez les éléments des lignes de la matrice a

Exercices

4.transformez la matrice de dimension(n), en matrice de dimension (m,t)

5 Ajoutez une ligne et une colonne a la matrice(m,t)

6. Affichez la Somme des colonnes

7.Affichez la Somme des lignes

8.Affichez éléments minimaux par lignes ainsi que la moyenne des lignes

9.Affichez éléments maximaux par colonnes ainsi que la moyenne des colonnes

10.Affichez la transposée et la matrice symétrique de a

11. Affichez le déterminant de a, ainsi que le produit de a et d

3.4. Bibliothèque Pandas

Pandas, c’est quoi ?

Panel Data analysis). C est la librairie pandas implémente la classe DataFrame. Panda est construit directement sur numpy. Les données manipulées par pandas sont stockées dans des «tableaux » numpy. pandas se spécialise dans l'analyse de données en deux dimensions, ce qu'on appelle les DataFrame

C’est une structure de table, chaque colonne porte un nom et contient un seul type de données. Il existe une grande variété de méthodes pour créer un DataFrame

- Une des bibliothèques Python les plus utilisées pour la Data Science.
- Développé en 2008 par Wes McKinney
- Open source
- Implémenté à partir de C – d’où sa rapidité
- A introduit les objets DataFrame et Series
- Donnee traitee sous forme de tableau Dataframe ont souvent le format CSV: Comma Separated Value

Installation et quelques fonctionnalités

Installation

C est une bibliothèque de Python, donc il s installe avec la bibliothèque Pandas.

Certes, cette bilbiotheque peut installe a part via

via pip depuis PyPI : $ pip install pandas

Quelques notions

Un objet Séries: C est un tableau monodimensionnel. Il peut être mis à jour, que de nouveaux éléments peuvent être ajoutés et que des éléments existants peuvent être supprimés. Il peut contenir des données de tout type

Exemple de création de série et récupération d une série:

```
import pandas as pd
series = pd.Series(data=[111, 222, 3], index = ['un','deux','trois'])
print(series['un'])
```

On peut utiliser les fonctions: len, mean, max

.

Objet DataFrame:

La structure de données la plus utilisée dans un projet de Data Science. C'est un tableau avec des lignes et des colonnes, comme une table SQL ou une feuille de calcul Excel.

La table peut stocker en mémoire des données dans des formats différents. Offre une analyse de séries chronologiques très performantes. Un DataFrame contient une ou plusieurs Series

création à partir de tableaux numpy

```
n=5
d = np.ones((n,2))
pd.DataFrame(d)
```

Exemple 2:

```
n = 5
d = { 'One' : np.linspace(1,n,n), 'Two': np.linspace(1,n,n)**2}
pd.DataFrame(d)
```

	ColonneA	ColonneB	ColonneC	ColonneD
index0				
index1				
index2				
index3				
index4				

Création d un DataFrame

Il existe plusieurs façons de créer un DataFrame: à partir d'un tableau, d'un dictionnaire, d'une liste, à partir d'une Series ou même d'un autre DataFrame. Je crée généralement un DataFrame à partir d'une Series:

```
import pandas as pd
d = {'ColonneA' : pd.Series([111, 222, 333]),
'ColonneB' : pd.Series([444, 555, 666])}
df = pd.DataFrame(d)
data = df['ColonneA']
data
```

- **Sélection de colonne(s)**

le nom de la colonne pour obtenir les données :

data = df['ColonneA']

Data

0 111

1 222

2 333

Name: ColonneA, dtype: int64

Pours supprimer une colonne, on utilise la fonction **pop**

- **Sélection de ligne(s)**

Si une ligne a une étiquette, nous pouvons utiliser la fonction loc :

data = df.loc**[**'label'**]** # Colonnes multiples, on aura

data = df.loc**[[**'label1', 'label2'**]]**

ou les ligne sont étiquetées,

data = df.iloc**[**0**]** # retourne la ligne d'indice 0 ou position 1

Data

Pour supprimer une ligne, on utilise la fonction **drop**.

Pour le cas precedent , nous aurons:

ColonneA 111

ColonneB 444

Name: 0, dtype: int64

Dans un DataFrame, nous pouvons appliquer un certain nombre de routines, par exemple, les colonnes peuvent être fusionnées, on peut appliquer des calculs mathématiques sur les lignes et/ou colonnes, on peut travailler sur des sous-ensembles du DataFrame.

Parcourir les éléments du dataframe

Pour parcourir les colonnes :

```
for column,items in df.iteritems():
print(column, items)
ColonneA
 0 111
1 222
2 333
Name: ColonneA, dtype: int64
ColonneB
0 444
1 555
2 666
Name: ColonneB, dtype: int64
```

Pour parcourir les lignes :

```
for index_of_row, row in df.iterrows():
print(index_of_row, row)
0 ColonneA 111
ColonneB 444
Name: 0, dtype: int64
1 ColonneA 222
ColonneB 555
Name: 1, dtype: int64
2 ColonneA 333
ColonneB 666
Name: 2, dtype: int64
```

Quelques fonction de pandas

Tries:

Par lignes: sorted_dataframe = dataframe.sort_index**()**

Par colonnes: sorted_dataframe = dataframe.sort_values**(**by='ColonneA'**)**

Lecture du fichier csv et à en créer un DataFrame :

import pandas as pd

dataFrame =pd.read_csv("fichier.csv",index_col=['ColonneA'])

Lecture du fichier Excel dans un DataFrame:

dataFrame = pd.read_excel**(**'fichier_Excel.xlsx', index_col=**[**'ColonneA'**])**

Le chargement d un feuillet du fichier Excel dans un DataFrame : dataFrame = pd.read_excel**(**open**(**'fichier_Excel.xlsx', 'rb'**)**, sheet_name='Sheet1'**)**

Head() retourner les n premières lignes d'un DataFrame :

r = dataFrame.head**(**10**)** # retourne les 10 premières lignes du dataframe

tail(n) retourne les n dernières lignes d'un DataFrame :

r = dataFrame.tail**(**10**)** # retourne les 10 dernières lignes du dataframe

Transposée: Pour retourner la transposée ou vice versa, on utilise l'attribut T :

transposed = dataFrame.T

Describe():résume rapidement de votre DataFrame, donne (moyenne, écart type, minimum, maximum et d'autres informations pour chacune des colonnes du DataFrame), on utilise la méthode *describe()* :

dataFrame.describe**()**

- **abs*()*** – valeurs absolues
- **mean*()*** – Valeurs moyennes. On a aussi *median()* et *mode()*
- **min()** – valeur minimale (et *max()*)
- **count*()***, **std*()*** – écart type, **prod*()*** – pour calculer le produit des valeurs et **cumsum*()*** pour calculer les sommes cumulées, etc…

dataFrame.notnull(): verification des valeurs manquantes

dataFrame.dropna(): suppression des valeurs manquantes

Comparaison des pourcentages:

- dataFrame.pct_change() # par colonne
- dataFrame.pct_change(axis=1) # par ligne

Covariance entre 2 colonnes:

dataFrame.cov() # entre toutes les colonnes

dataFrame**[**'colonneA'**]**.cov**(**dataFrame**[**'colonneB'**])** # entre 2 colonnes

Corrélation:

dataFrame.corr() # entre toutes les colonnes

dataFrame**[**'colonneA'**]**.corr**(**dataFrame**[**'colonneB'**])** # entre 2 colonnes

Grouper les lignes ou colonnes: ***groupby()*** **:**

groupedDataFrame = dataFrame.groupby**(**'NomColonne'**)**

Filtrer:

dataFrame.filter(myCustomFunction)
#myCustomFunction prend un paramètre et renvoie une valeur

- Fusionner: Pandas propose des fonctionnalités permettant de joindre plusieurs DataFrames sur la base de jointures gauche, droite, intérieure et extérieure. On utilise la fonction ***merge()***. le paramètre **on** définissant les colonnes à joindre et le paramètre **how** décrivant le type de jointure : ***left*** (gauche), ***right*** (droite), ***outer*** (extérieur) ou ***inner*** (intérieur).
- **concat()** : elle sert a concaténer 2 DataFrames

Exemple: pd.concat**([**df1, df2**])**

- **Représentation graphique:**
- **dataFrame.plot.bar()** # retourne un diagramme à barres
- **dataFrame.diff.hist(bins=10)** # retourne un histogramme
- **dataFrame.plot.scatter()** # retourne un graphique de nuage de points

Chap.3. Interface graphique en Python

Pour interagir avec le système , l'utilisateur a besoin de l interface utilisateur. L interface peut être graphique, web, console ou mobile. Le python, nous permet de faire li interface graphique de l utilisateur(GUI: Graphical User Interface) à l'aide d'un module présent par défaut dans Python : **Tkinter.**

Ce module permet de créer des interfaces graphiques en offrant une passerelle entre Python et la bibliothèque Tk. On pourra pouvoir à créer des fenêtres, créer des boutons, faire réagir nos objets graphiques à certains évènements…

Le module Tkinter

Tkinter (Tk interface) est un module intégré à la bibliothèque standard de Python, bien qu'il ne soit pas maintenu directement par les développeurs de Python.

Il offre un moyen de créer des interfaces graphiques via Python. Tkinter est disponible sur Windows et la plupart des systèmes Unix. D ou l application window creee est portable.

Il existe d autres d'autres bibliothèques pour créer des interfaces graphiques.

Tkinter a l'avantage d'être disponible par défaut, sans nécessiter une installation supplémentaire

Les 15 widgets de base de Tkinter sont :

• Button : un bouton simple, qui permet de lancer une commande ;

• Canvas : un widget générique, qui offre une surface de dessin ;

• Checkbutton : une case à cocher ; • Entry : un champ texte ;

• Frame : un widget qui peut contenir d'autres widgets (une fiche) ;

• Label : affiche un texte ou une image ;

• Listbox : une liste de choix déroulante ;

• Menu : un menu ;

• Menubutton : un élément de menu, qui permet de lancer une commande ;

• Message : un label évolué ;

• Radiobutton : un sélecteur ;

• Scale : une réglette qui permet de modifier une valeur ;

• Scrollbar : un ascenseur, généralement associé à la bordure d'un autre widget pour se déplacer ;

• Text : peut contenir du texte éditable, et des éléments supplémentaires comme du texte ;

• Toplevel : équivalent au widget Frame, mais permet de gérer une fenêtre modale

exemple

Le code minimal pour la creation de la page windows est la suivante

```
from tkinter import *
fenetre = Tk()  # On crée ensuite un objet de la classe Tk.
champ_label = Label(fenetre, text="Salut  a tous et toutes  !")
champ_label.pack()  # On affiche le label dans la fenêtre
fenetre.mainloop() # On démarre la boucle Tkinter qui s'interompt quand on ferme la fenêtre
```

Les objets graphiques (boutons, champs de texte, cases à cocher, barres de progression…) sont appelés des widgets

Il existe d'autres options communes à la plupart des widgets (la couleur de fond bg, la couleur du widget fg, etc.) et d'autres plus spécifiques à un certain:type de widget. Le Label par exemple possède l'option text représentant le texte affiché par le Label.

Les boutons

Les boutons sont des widgets sur lesquels on peut cliquer et qui peuvent déclencher des actions ou commandes comme nous le verrons ultérieurement plus en détail

```
from tkinter import *
fenetre = Tk()
champ_label=Label(fenetre, text="Salut  a tous et toutes  !")
bouton_quitter=Button(fenetre,
text="Quitter",command=fenetre.quit)
champ_label.pack()
bouton_quitter.pack()
fenetre.mainloop()
```

Une ligne de saisie

En fait de zone, ils'agit d'une ligne simple. On préférera créer une variable Tkinter associée au champ de texte.

```
var_texte = StringVar()

ligne_texte = Entry(fenetre, textvariable=var_texte, width=30)

ligne_texte.pack()
```

LES CASES À COCHER

Les cases à cocher sont définies dans la classe Checkbutton. Pour surveiller l'état d'une case à cocher (qui peut être soit active soit inactive), on préférera créer une variable de type IntVar plutôt que StringVar, bien que ce ne soit pas une obligation.

```
from tkinter import *
fenetre = Tk()
champ_label = Label(fenetre, text="Salut  a tous et toutes
!")
bouton_quitter=Button(fenetre,
text="Quitter",command=fenetre.quit)
var_texte = StringVar()
ligne_texte = Entry(fenetre, textvariable=var_texte,
width=30)
champ_label.pack()
bouton_quitter.pack()
ligne_texte.pack()
fenetre.mainloop()
```

```
from tkinter import *
fenetre = Tk()
champ_label = Label(fenetre, text="Salut  a tous et toutes  !")
bouton_quitter=Button(fenetre, text="Quitter",command=fenetre.quit)
var_texte = StringVar()
ligne_texte = Entry(fenetre, textvariable=var_texte, width=30)
champ_label.pack()
bouton_quitter.pack()
ligne_texte.pack()
fenetre.mainloop()
```

```
var_case = IntVar()
case = Checkbutton(fenetre, text="Ne plus poser cette
question",variable=var_case)
case.pack()
```

Code global

```
from tkinter import *
fenetre = Tk()
champ_label = Label(fenetre, text="Salut  a tous et toutes  !")
bouton_quitter = Button(fenetre, text="Quitter",command=fenetre.quit)
var_texte = StringVar()
ligne_texte = Entry(fenetre, textvariable=var_texte, width=30)
var_case = IntVar()
case = Checkbutton(fenetre, text="Ne plus poser cette
question",variable=var_case)

case.pack()
ligne_texte.pack()
champ_label.pack()
bouton_quitter.pack()
fenetre.mainloop()
```

Les boutons radio

Les boutons radio (radio buttons en anglais) sont des boutons généralement présentés en groupes.
Quand vous cliquez sur l'un des boutons, celui-cise sélectionne et tous les autres boutons du même groupe se désélectionnent.

```
var_choix = StringVar()
choix_rouge = Radiobutton(fenetre, text="Rouge", variable=var_choix, value="rouge")
choix_vert = Radiobutton(fenetre, text="Vert", variable=var_choix, value="vert")
choix_bleu = Radiobutton(fenetre, text="Bleu", variable=var_choix, value="bleu")
 choix_rouge.pack()
choix_vert.pack()
choix_bleu.pack()
```

Les listes déroulantes

Ce widget permet de construire une liste dans laquelle on peut sélectionner un ou plusieurs éléments.

```
from tkinter import *
fenetre = Tk()
liste = Listbox(fenetre)
liste.insert(END, "Pierre")
liste.insert(END, "Feuille")
liste.insert(END, "Ciseau")
liste.pack()
```

canvas

```
from tkinter import *
fenetre = Tk()
canvas = Canvas(fenetre, width=150, height=120,
background='yellow')
ligne1 = canvas.create_line(75, 0, 75, 120)
ligne2 = canvas.create_line(0, 60, 150, 60)
txt = canvas.create_text(75, 60, text="Cible",
font="Arial 16 italic", fill="blue")
canvas.pack()
```

Scale et le spinbox

```
from tkinter import *
fenetre = Tk()
value = DoubleVar()
scale = Scale(fenetre, variable=value)
scale.pack()
canvas.pack()
```

```
from tkinter import *
fenetre = Tk()
s = Spinbox(fenetre, from_=0, to=10)
s.pack()
canvas.pack()
```

Opération arithmétique avec gui

```
import os
import tkinter
import sys
import tkinter as tk
def somme():
    n1= int(nb1.get())
    n2=int(nb2.get())
    som= n1+n2
    nb3.insert(0,som)
fenetre =tkinter.Tk()
nb1=tk.Entry(fenetre)
nb1.pack()
nb2 =tk.Entry(fenetre)
nb2.pack()
bouton=tk.Button(fenetre,text="Sommation", command=somme)
bouton.place(x=120,y=100)
bouton.pack()
nb3 =tk.Entry(fenetre)
nb3.pack()
fenetre.minsize(400,300)
fenetre.mainloop()
```

```
import os
import os
import tkinter
import sys
import tkinter as tk
def expression():
    n1= int(nb1.get())
    n2=int(nb2.get())
    n3=int(nb3.get())
    express=(n1+n2)*n3
    nb4.insert(0,express)
fenetre =tkinter.Tk()
nb1=tk.Entry(fenetre)
nb1.pack()
nb2 =tk.Entry(fenetre)
nb2.pack()
nb3 =tk.Entry(fenetre)
nb3.pack()
bouton=tk.Button(fenetre,text="Expression", command=expression)
bouton.place(x=120,y=100)
bouton.pack()
nb4 =tk.Entry(fenetre)
nb4.pack()
fenetre.minsize(400,300)
fenetre.mainloop()
```

Représentation des données avec Matplotlib

```
import matplotlib.pyplot as plt
plt.plot([1,2,3,4])
plt.ylabel('Label 1')
plt.show()
```

Il est possible de superposer les courbes:

```
import matplotlib.pyplot as plt
plt.plot([50,100,150,200], [1,2,3,4])
plt.plot([50,100,150,200], [2,3,7,10])
plt.plot([50,100,150,200], [2,7,9,10])
plt.show()
```

Références

- https ://sqlpro.developpez.com/SGBDR/ReglesCodd/
- https://www.createursdemondes.fr/2016/05/limites-des-sgbdr-pour-le-big-data-nosqlet-newsql/
- https://stph.scenari-community.org/idl-bd/idl-nosql/co/nos01.html
- https ://www.lebigdata.fr/apache-cassandra-definition
- https ://www.ambient-it.net/top-meilleures-db-nosql-2019/
- https://firebase.google.com/docs/firestore/manage-data/data-types
- https://firebase.google.com/docs/firestore/manage-data/structure-data
- W. Mac Kinney, Python for Data Analysis, O'Reilly, 2013, http:// it-ebooks.info/book/1041/
- http://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.html#pandas.DataFrame), c
- https://perso.univ-lyon1.fr/marc.buffat/COURS/BOOK_OUTILSNUM_HTML/MGC2367M/cours/Chap3_PandasBigdata.html
- pandas.pydata.org